

title Multi-Tenancy & White-Label
subtitle Organizations, brand-scoped data, theme/asset overrides, and SCIM for the greenroom platform
date 2026-05-17

Multi-Tenancy & White-Label

Status: planning. Identity ships single-tenant today (per `CLAUDE.md` §Multi-tenancy + SCIM and `docs/chat-spec.md` §P1). This doc is the **target spec** for org-aware identity, org-scoped data across apps, and per-org white-labeling. The code may not match it in some places; this is what the build converges to.

The greenroom platform is **B2B2C**: cannabis brands pay for a presence on the platform; budtenders (retail-store staff at dispensaries) are the audience the brands are paying to reach. Cannabis advertising is heavily regulated, so brands engage budtenders through SaaS-mediated relationships rather than traditional ads.

- **Brand** = paying enterprise customer = an **organization**. Has its own admins, its own chat rooms, its own theme, eventually its own subdomain. A brand's staff are members of the brand's org.
- **Budtender** = ordinary platform user. Authenticates against guestlist the same way anyone else does, but **does not belong to any org**. Budtenders use the platform's public surfaces (public chat rooms, DMs, account settings) and interact with brand-owned surfaces (org chat rooms they self-join, brand-published content) without ever holding an org membership.
- A user can belong to **zero, one, or many** orgs. The dominant case is zero (budtenders are the majority); the paying case is one or many (brand staff). The platform operator runs the spine.

Retailers (dispensaries themselves, as opposed to the brands selling in them) may become a second org type later — likely as a separate `organizationType` discriminator on the same `organization` table, with a different role set and a different relationship to budtenders (who work for retailers). Out of scope for v1; pencilled in §14 as an open question.

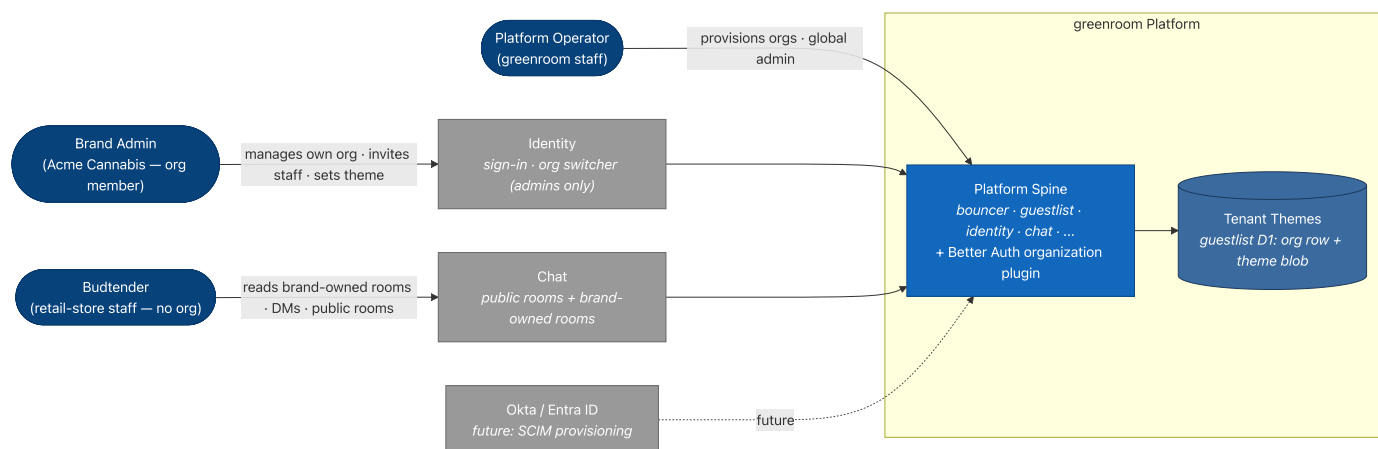
v1 scope. Operator-driven onboarding, no custom RBAC. The platform admin (a single greenroom-staff user with `user.role === "admin"`) creates orgs and invites/adds the first members from identity's existing `/admin/*` surface. Org admins (`member.role === "admin" | "owner"`) can only configure their org's theme + members. Three BA built-in roles (`owner / admin / member`); no custom role definitions; no teams; no dynamic AC; no SCIM. Everything in §11, §13 (deferred parts), and §6.3–6.4 lands later. The goal is **a demo path the operator can drive manually** as fast as possible.

This doc covers four entangled concerns:

1. **Org identity** — Better Auth's `organization` plugin wired in `services/guestlist/`. The data model, the API surface apps consume, the role model.
2. **Org context propagation** — how an "active org" reaches an app request: cookie → guestlist → envelope → app. What the bouncer attestation carries, what apps trust, what apps re-verify.
3. **Org-scoped data in apps** — chat rooms tagged with `ownerOrgId`, future transfers/inventory/etc. apps following the same pattern. How an app asks "is this user an admin of this org?" without trusting the client.
4. **White-label** — per-org theme tokens, logo/wordmark, OG images, eventually per-org domains. The override layer lives over `packages/design/`'s token system; no per-fork code edits needed for a new brand.

SCIM is a fifth concern (enterprise identity sync from Okta/Entra). It's out of scope for the first cut and pencilled in §11 as a follow-on plugin.

§1 — Context



What changes vs. docs/ARCHITECTURE.md .

The platform's existing primitives — bouncer attestation envelope, guestlist as session authority, app→guestlist service binding — remain unchanged. This spec **layers on top**:

- Guestlist gains the BA `organization` plugin tables and routes.
- The bouncer envelope grows two optional fields: `activeOrgId` and `activeOrgRole` .
- Apps gain a `platform.getActiveOrg()` accessor analogous to `platform.getEnvelope()` .
- A new guestlist endpoint `/u/theme/:orgId` serves per-org theme overrides, consumed by every app's root layout the way `/u/avatar/:refId` is consumed by user-rendering code today.
- A new identity surface (`/orgs` , `/orgs/:slug/settings` , `/orgs/:slug/theme`) hosts org admin UI.

Everything else — auth flow, session refresh, envelope verification, log correlation — is unchanged.

What this is not.

- Not row-level multi-tenancy in the database sense (no `tenant_id` prefix on every table). Orgs are a layer over user identity; apps that host org-scoped data tag their rows with `ownerOrgId` explicitly.
- Not full data isolation across orgs (no separate D1 per tenant). Plausible in a far-future "enterprise on-prem" tier; out of scope here.
- Not a fork-per-tenant model. One platform spine, many tenants, one release cadence.

§2 — Concepts

§2.1 Organization

A platform-level entity owned by a brand. Created either by a platform operator (provisioning a new customer) or — once we trust the funnel — by self-signup. Each org has:

Field	Meaning
<code>id</code>	ULID. Opaque, stable. Used everywhere downstream as <code>ownerOrgId</code> .
<code>slug</code>	URL-safe handle. Public-ish; appears in subdomain / path. Lowercased, dash-separated. Globally unique.
<code>name</code>	Human-readable. Appears in invites, account switcher, OG titles.
<code>logo</code>	URL into roadie (same broker as user avatars). Optional.
<code>metadata</code>	JSON blob. Extension point — theme overrides live here in v1; promote out if it grows.
<code>createdAt</code>	When the org was provisioned.

`metadata` is the seam for everything we don't want to schema-migrate yet: theme tokens, billing plan, feature flags, contact info. Promote any of these into typed columns once they stabilize.

§2.2 Membership

A user can belong to zero, one, or many orgs. Membership is a join row: `(userId, organizationId, role)`. Role is one of `owner` | `admin` | `member` (BA's built-in trio); v1 ships with these three only (no custom roles, no dynamic AC — see §6.3 for the future story).

Importantly: **org membership ≠ app-internal role**. Org members are exclusively brand staff. Budtenders are not org members — they interact with brand-owned surfaces (e.g. an org-owned chat room) the same way any authenticated user does, subject to chat's per-room role table (`owner` / `admin` / `mod` / `member` / `guest`, per `docs/chat-spec.md` §1.3). Org membership only authorizes **brand-side admin** operations on the brand's own org — creating that brand's chat rooms, configuring that brand's theme, managing that brand's staff roster.

§2.2.1 Users without orgs (the majority case)

A user with **zero memberships** is the *default* user, not an edge case. The platform is the union of three populations:

- **Budtenders** (zero memberships, `user.role === "user"`). The B2C audience. Sign up, use public chat rooms, hold DMs, interact with brand-owned chat rooms they self-join. Quantitatively the majority of users; product gravity centers here.
- **Brand staff** (one or more memberships, `user.role === "user"`). The B2B revenue side. Members of one or more org rows. May hold `owner` / `admin` / `member` roles in their orgs.
- **Operators** (typically zero memberships, `user.role === "admin"`). Greenroom staff. Authority is platform-wide; orgs are resources they manage, not surfaces they belong to.

A user can transition between these populations:

- A budtender invited as brand staff gains a membership and becomes brand staff for that org (still also a budtender; the categories aren't exclusive).
- A brand admin removed from their org loses that membership and reverts to budtender for that brand's surfaces.

What this means in practice:

- `envelope.activeOrg` is `null` for zero-membership users; that's the signed claim, not an error. **The hot path is `activeOrg === null`** — apps must render correctly for that case first.
- Identity's UI gracefully degrades: the org switcher only appears when the user has memberships or pending invitations. For budtenders the header looks identical to a single-tenant app.
- App shells (chat, identity) work fully without an active org. Only surfaces that are *inherently brand-scoped* (creating an org room, editing a brand's theme, the brand's admin surfaces) require one.
- Public surfaces (public chat rooms, account settings, DMs) treat all authenticated users the same regardless of org status.

This shape — "user identity is universal, org membership is a layer over it" — is what makes the B2B2C model fit the platform's existing single-tenant primitives without contortions.

§2.3 Active organization

A user with multiple memberships has, at any moment, one **active org** — the org whose data they're currently looking at. This is per-session, not per-user: tab 1 can have Acme active, tab 2 can have BetaCo active, both from the same login.

Storage of `activeOrganizationId`:

- **Authoritative copy:** BA's `session` row (the plugin adds the column).
- **Bouncer envelope:** carries a copy as `activeOrgId` so apps can read it without a guestlist hop (§4).
- **No cookie:** switching active org goes through guestlist's `setActiveOrganization` endpoint, which writes the session row and the BA cookie cache; the next request reflects it.

Apps treat `activeOrgId` the same way they treat the actor: a signed, short-lived claim from the envelope. Authoritative checks (am I really an admin of this org?) still resolve through guestlist on the request path that needs them.

§2.4 Tenant theme

A small JSON blob of CSS custom-property overrides keyed by **shadcn semantic token names** (`--color-primary`, `--color-background`, etc. — see §8.1.3 for the full allowlist). Lives at `org.metadata.theme` in the BA org table; cached in front by guestlist; served to apps as a CSS rule at page render time.

Tenants do **not** override greenroom's internal palette names (`--color-glyph`, `--color-blood`, etc.) — those are an implementation detail of greenroom's own default theme and may change between releases. Tenants only ever speak shadcn's standardized semantic token surface, which `packages/design/src/theme.css` already maps onto the internal palette (§8.1.2).

A tenant theme has three pieces:

1. **Color override map** — `{ "--color-primary": "#7c3aed", ... }`. Sparse; any token not overridden falls through to the platform's own default theme (whatever the internal palette is at the time). Has parallel light/dark sub-maps. Keys validated against the shadcn allowlist.
2. **Logo asset** — roadie reference to an SVG/PNG logo. Replaces the greenroom wordmark in app chrome when an org is active and the surface is org-scoped (org-owned chat room, identity org-settings page).
3. **Wordmark text** — short string (1-3 words). Replaces `platformConfig.brand.name` in the same places.

§2.5 Domain strategy

Two viable options, picked at deploy time per fork. The platform supports both shapes; only routing config differs.

Strategy	Public URL	Pros	Cons
Subdomain per org	<code>acme.greenroom.app</code>	Cleanest brand feel. Each org gets a "site". Custom domains (<code>chat.acme.com</code>) layer in naturally.	Bouncer ROUTES grows per-org. CF Custom Domains have a per-zone limit (~100 in standard plans; 1000+ for enterprise). DNS automation needed for self-serve.
Path per org	<code>greenroom.app/o/acme/chat</code>	One CF Custom Domain. No DNS automation. Org switching is just a route change.	Cookies span all orgs (already the case, this is fine), but URLs look less branded. Custom domains require an extra layer.

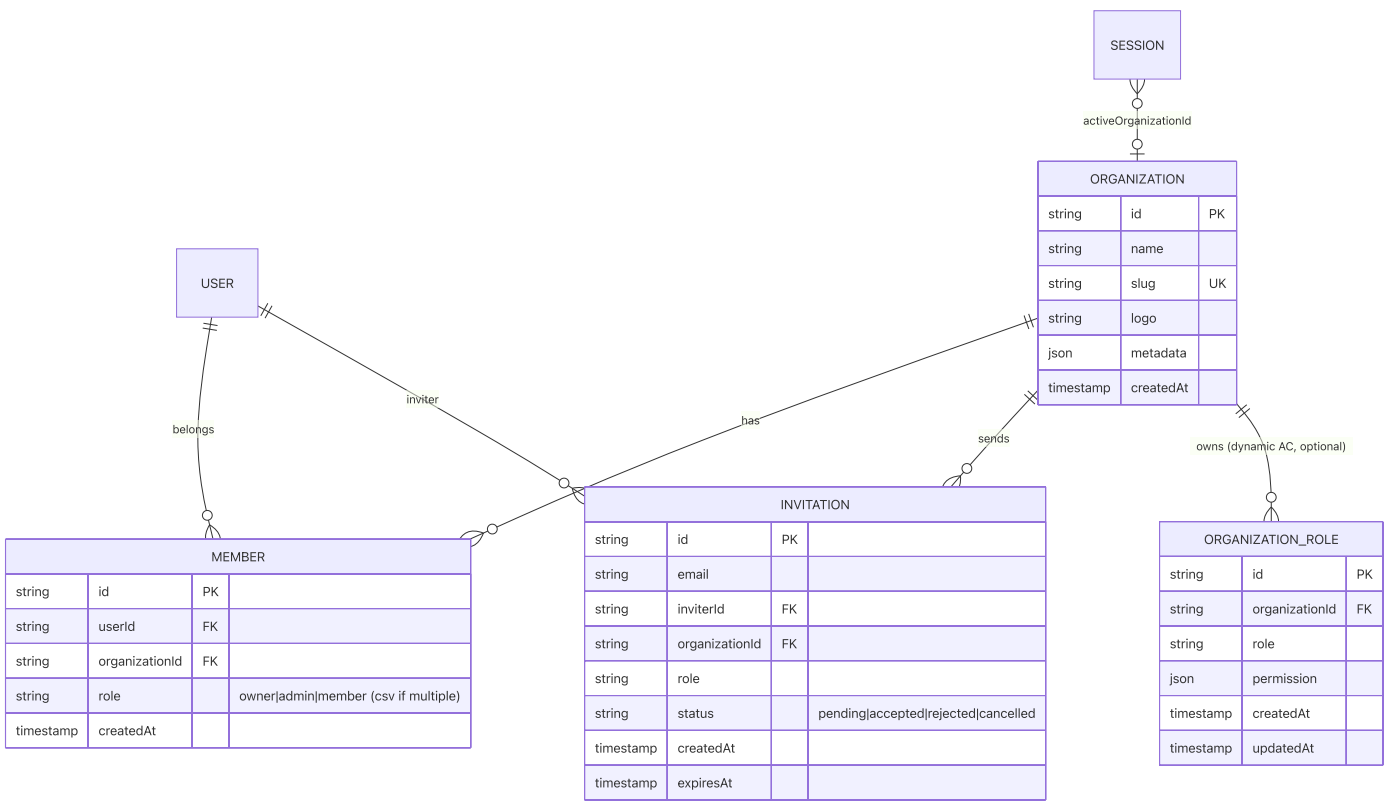
Recommendation: start with path-per-org for v1, design data and APIs to be subdomain-ready. Cutover to subdomain-per-org happens by changing bouncer's `ROUTES` and registering CF Custom Domains; no app code changes because nothing in app code parses the host for org identity (that role belongs to the envelope's `activeOrgId`).

Custom domain support (`chat.acme.com` → bouncer → chat app, with Acme as activeOrg) is a separate concern handled in §10.3.

§3 — Schema

The BA organization plugin defines the tables; we extend them in two places: typed Drizzle mirrors in `services/guestlist/src/schema.ts` for our own queries, and additional `metadata` typing.

§3.1 Tables added by the plugin



§3.2 Modifications to existing tables

The plugin adds one column to `session` :

```
ALTER TABLE session ADD COLUMN active_organization_id TEXT;
```

If teams are enabled later (we are not enabling teams in v1; see §6.4), a second column appears:

```
ALTER TABLE session ADD COLUMN active_team_id TEXT;
```

No changes to `user` , `account` , `twoFactor` , `passkey` , `apikey` , or the `oauth_*` tables.

§3.3 Metadata typing

```
// services/guestlist/src/schema.ts (target – added next to existing tables)
export const organization = sqliteTable("organization", {
  id: text("id").primaryKey(),
  name: text("name").notNull(),
  slug: text("slug").notNull().unique(),
  logo: text("logo"),
  metadata: text("metadata", { mode: "json" }).$type<OrgMetadata>(),
  createdAt: integer("created_at", { mode: "timestamp_ms" }).notNull(),
});

export type OrgMetadata = {
  /** Per-org theme override. Null/missing = inherit platform default. */
  theme?: TenantTheme;
  /** Billing plan key. Null/missing = "trial". */
  plan?: "trial" | "starter" | "pro" | "enterprise";
  /** Feature flag overrides. Sparse. */
  flags?: Record<string, boolean>;
  /** Operator-internal notes. Never shown to org users. */
  operatorNotes?: string;
};

export type TenantTheme = {
  /**
   * Sparse map of shadcn semantic token name → CSS value, light mode.
   * Keys are restricted to SHADCN_TENANT_TOKEN_ALLOWLIST (see §8.1.3).
   * Tokens not in the allowlist are rejected by the editor and stripped
   * by `<OrgThemeStyle>` even if persisted.
   */
  light?: Partial<Record<ShadcnTenantToken, string>>;
  /** Same shape as `light`, applied under `[data-theme="dark"]`. */
  dark?: Partial<Record<ShadcnTenantToken, string>>;
  /** Roadie reference for the org's logo asset (SVG preferred). */
  logoRefId?: string;
  /** Wordmark text override. Defaults to organization.name when unset. */
  wordmark?: string;
  /**
   * Schema version. Bumps when the allowlist changes shape (e.g. shadcn
   * adds new semantic tokens, or we promote `--font-*` into the allowlist).
   * Migration scripts read this to rewrite stored tenant themes safely.
   */
  schemaVersion: 1;
};

export type ShadcnTenantToken =
  | "--color-background"
  | "--color-foreground"
  | "--color-card"
  | "--color-card-foreground"
  | "--color-popover"
  | "--color-popover-foreground"
  | "--color-primary"
  | "--color-primary-foreground"
  | "--color-secondary"
  | "--color-secondary-foreground"
  | "--color-muted"
  | "--color-muted-foreground"
  | "--color-accent"
  | "--color-accent-foreground"
  | "--color-destructive"
  | "--color-border"
  | "--color-input"
  | "--color-ring"
  | "--color-chart-1"
  | "--color-chart-2"
```

```
| "--color-chart-3"  
| "--color-chart-4"  
| "--color-chart-5"  
| "--color-sidebar"  
| "--color-sidebar-foreground"  
| "--color-sidebar-primary"  
| "--color-sidebar-primary-foreground"  
| "--color-sidebar-accent"  
| "--color-sidebar-accent-foreground"  
| "--color-sidebar-border"  
| "--color-sidebar-ring"  
| "--radius";
```

Why store theme in `metadata` rather than its own table:

- We don't know the final shape yet. JSON blob in `metadata` lets the schema evolve without migrations.
- The theme is read on every page render. Storing alongside the org row means one D1 lookup, not a join.
- BA's plugin already exposes `metadata` reads/writes; we get the API for free.

Once theme stabilizes (after the first 2-3 paying orgs have configured non-trivial themes), promote it to a `tenant_theme` table with a typed schema. Migration is mechanical.

§4 — Plugin wiring

The BA organization plugin lands in `packages/auth/src/server.ts`'s plugin list (the canonical platform factory). Guestlist picks it up automatically via `createPlatformAuth`; no per-service wiring.

§4.1 Factory addition

```
// packages/auth/src/server.ts (target – added to the existing plugin array)
import { organization } from "better-auth/plugins";
import { createAccessControl } from "better-auth/plugins/access";
import {
  defaultStatements,
  adminAc,
  memberAc,
  ownerAc,
} from "better-auth/plugins/organization/access";

const orgStatement = {
  ...defaultStatements,
  // Brand-platform-specific resources. Apps reference these on their
  // server side as `await platform.getGuestlist().hasOrgPermission({
  //   permissions: { theme: ["update"] }
  // })`.
  theme: ["update"],
  billing: ["read", "update"],
  scim: ["read", "configure"],
} as const;

const ac = createAccessControl(orgStatement);

const memberRole = ac.newRole({ ...memberAc.statements });
const adminRole = ac.newRole({
  ...adminAc.statements,
  theme: ["update"],
  billing: ["read"],
});
const ownerRole = ac.newRole({
  ...ownerAc.statements,
  theme: ["update"],
  billing: ["read", "update"],
  scim: ["read", "configure"],
});

// ... inside the plugins array:
organization({
  ac,
  roles: {
    owner: ownerRole,
    admin: adminRole,
    member: memberRole,
  },
  creatorRole: "owner",
  // v1: operator-provisioned. End-users cannot create orgs from
  // identity's normal UI. The operator-only `/admin/orgs/new` route
  // bypasses this by calling the BA server API *without* session headers
  // and passing `userId` explicitly (see §4.5 – Operator provisioning).
  allowUserToCreateOrganization: false,
  // No invitationLimit; defaults (100) is fine.
  invitationExpiresIn: 60 * 60 * 24 * 7, // 7 days
  cancelPendingInvitationsOnReInvite: true,
  requireEmailVerificationOnInvitation: true,
  organizationHooks: {
    afterCreateOrganization: async ({ organization }) => {
      // Audit log entry. Defer to background tasks so org creation isn't
      // blocked by log write.
      opts.backgroundTasks?.handler?.(
        log.info("organization.created", {
          orgId: organization.id,
          slug: organization.slug,
        })
      ),
    },
  },
});
```



```

    },
    beforeCreateInvitation: async ({ invitation, organization }) => {
      // Stamp the org's name onto the invitation so promoter's templates
      // can render brand-aware copy without re-querying guestlist.
      return {
        data: { ...invitation, metadata: { orgName: organization.name } },
      };
    },
  },
  async sendInvitationEmail(data) {
    // promoter wires this; see §4.3.
  },
});

```

§4.2 Active-org bootstrapping

The plugin docs note that `activeOrganizationId` defaults to `null` on fresh sessions. For users who belong to exactly one org, we auto-set it on session creation via a database hook — saves the round-trip to call `setActive` after every sign-in:

```

// packages/auth/src/server.ts (added to createPlatformAuth's databaseHooks)
databaseHooks: {
  session: {
    create: {
      before: async (session) => {
        // Look up the user's memberships; if exactly one, set it active.
        // Done in BA's session-create transaction so it's atomic with the
        // session insert.
        const memberships = await opts.database
          .findMany({ model: "member", where: ["userId", session.userId] })
          .catch(() => [] as Array<{ organizationId: string }>);
        if (memberships.length === 1) {
          return {
            data: { ...session, activeOrganizationId: memberships[0].organizationId },
          };
        }
        return { data: session };
      },
    },
  },
},

```

For users with multiple memberships, `activeOrganizationId` stays `null` on first session and is set explicitly when the user picks an org in the account switcher.

§4.3 Invitation email path

Promoter (the platform's outbound email worker) gains a new template `organization-invitation`. The signature mirrors existing kinds:

```
// services/promoter/src/templates.ts (target – added to the existing union)
export type PromoterPayload =
  | /* ... existing kinds ... */
  | {
    kind: "organization-invitation";
    to: { email: string; name?: string };
    inviterName: string;
    inviterEmail: string;
    organizationName: string;
    inviteUrl: string;
    from: string;
  };

```

Guestlist's `sendInvitationEmail` callback packs the BA-supplied invitation data into a promoter call:

```
async sendInvitationEmail(data) {
  await env.PROMOTER.send(
    {
      kind: "organization-invitation",
      to: { email: data.email },
      inviterName: data.inviter.user.name,
      inviterEmail: data.inviter.user.email,
      organizationName: data.organization.name,
      inviteUrl: `${env.IDENTITY_URL}/orgs/accept/${data.id}`,
      from: env.EMAIL_FROM,
    },
    promoterMeta(),
  );
},

```

§4.4 Operator provisioning

The BA plugin has one critical quirk: `auth.api.createOrganization(...)` attaches the org to the caller's session user unless you call it **server-side without session headers** and pass `userId` explicitly. The docs phrase it as:

*"With session headers: The organization is created for the authenticated session user. The `userId` field is **silently ignored**."*

This is the seam the operator-creates-org-for-someone-else flow rides on (see also the O-1 / O-4..O-7 operator checkpoints in §12). Identity's operator-only `/admin/orgs/new` server function:

```
// apps/identity/src/routes/admin.orgs.new.tsx (target)
import { createServerFn } from "@tanstack/react-start";
import { platform } from "@lib/platform";

export const createOrgAsOperator = createServerFn({ method: "POST" })
  .validator((data: { name: string; slug: string; ownerUserId: string }) => data)
  .handler(async ({ data, context }) => {
    // Gate: operator only. `actor.role === "admin"` is the platform
    // admin role from BA's `admin()` plugin (already wired).
    const env = await platform.getEnvelope(context.request.headers);
    if (env?.actor?.role !== "admin") throw new Error("forbidden");

    // Call guestlist's underlying BA api directly via the service
    // binding, NOT through cookies. The Eden treaty client's `auth`
    // surface forwards cookies; we instead use guestlist's typed RPC
    // for "admin actions on behalf of a user" – call without cookies
    // and pass `userId`.
    const result = await platform.getGuestlist().createOrgAsOperator({
      name: data.name,
      slug: data.slug,
      ownerUserId: data.ownerUserId,
    });
    return result;
  });
```

The matching guestlist client helper (added in §6.2):

```
async createOrgAsOperator(args: { name: string; slug: string; ownerUserId: string }) {
  // Note: NOT using the cookie-forwarding `auth` client. The Eden API
  // surface lets us call guestlist's underlying BA api without session
  // headers, which makes the userId param meaningful.
  return await api.admin.orgs.create.post({
    name: args.name,
    slug: args.slug,
    ownerUserId: args.ownerUserId,
  });
},
```

And the guestlist endpoint:

```
// services/guestlist/src/index.ts (target – new operator route)
.post(
  "admin/orgs/create",
  async ({ body }) => {
    return await auth.api.createOrganization({
      body: {
        name: body.name,
        slug: body.slug,
        userId: body.ownerUserId, // becomes owner per `creatorRole: "owner"`
      },
      // intentionally no `headers:` – that's the trigger
    });
  },
  { admin: true /* uses the existing admin Elysia macro */ },
)
```

Two flows fall out of this:

1. **Operator creates org owned by an existing user.** The user already signed up (maybe via an out-of-band invitation, maybe via self-signup). Operator provides their `userId` (or email → user lookup) on the new-org form. New org is created with that user as `owner`. Operator can also be `admin` if desired (separate `addMember` call, same operator route).

2. **Operator creates org without a designated owner yet.** Operator passes their own `userId` as `ownerUserId`. Brand admin is added as `admin` once they sign up (via `addMember`), then promoted to `owner` when ready (via `updateMemberRole`). This is the path for "I have a new customer; they haven't created an account yet but I want to set up their org and invite them."

Same pattern for member adds: identity's `/admin/orgs/:id` view exposes "Add member by email" → guestlist looks up `user.email`, calls `auth.api.addMember({ body: { userId, organizationId, role }, /* no headers */ })`. If the email matches no existing user, the operator uses "Invite by email" instead, which goes through the standard BA invitation flow.

§4.5 Codegen + migrations

After the plugin lands:

```
cd services/guestlist
bun run codegen      # `bunx auth generate` against auth-config.ts
# review the generated migration; commit it
bun run migrate      # apply to local D1
```

The same migration runs against staging / production D1 instances via the existing wrangler `d1 migrations apply` flow (`scripts/migrate-d1.ts` if present, or by hand).

§5 — Session & envelope changes

The envelope (`docs/ARCHITECTURE.md §4.1.2`) is the spine's signed identity claim. Adding org context to it lets apps gate behavior without an extra guestlist hop on the read path.

§5.1 New envelope fields

```
// packages/auth/src/envelope/types.ts (target – added to EnvelopePayload)
export type EnvelopePayload = {
  v: 1;
  iss: "bouncer";
  iat: number;
  exp: number; // iat + 30s
  host: string;
  actor: EnvelopeActorUser | null;
  session: EnvelopeSessionData | null;
  /**
   * Active org for this session, as resolved at envelope-mint time.
   * Null when:
   * - the user has no memberships, or
   * - the user has multiple memberships and hasn't picked one yet, or
   * - actor is null (no session).
   * Invariant: `actor === null ⇒ activeOrg === null`.
   */
  activeOrg: EnvelopeActiveOrg | null;
};

export type EnvelopeActiveOrg = {
  id: string;
  slug: string;
  /** The user's role in this org, as BA computed it at envelope-mint time. */
  role: "owner" | "admin" | "member" | (string & {});
};
```

The minter (`services/bouncer/src/envelope.ts`) fills these from the session data returned by `guestlist.getSession()` . Guestlist's `getSession` return shape already carries `activeOrganizationId` once the plugin lands; projection into the envelope is a few extra lines in `services/bouncer/src/session.ts:toEnvelopeSession` .

Why role-in-envelope (not just id):

- Apps that need only "is this user an admin of their active org?" can short-circuit on the envelope. No guestlist RPC.
- The role is point-in-time. Like the rest of the envelope it has a 30s TTL; if the user is demoted mid-session, the envelope's claim becomes stale for up to 30 seconds, after which the refresh picks up the new role. Acceptable for read paths; write paths re-verify (§5.4).

§5.2 Verifier output

```
// packages/auth/src/envelope/verify.ts (target – extended verifier return)
type EnvelopeResult =
  | {
    kind: "valid";
    actor: EnvelopeActorUser | null;
    session: EnvelopeSessionData | null;
    activeOrg: EnvelopeActiveOrg | null;
  }
  | { kind: "missing" }
  | { kind: "invalid"; reason: string };
```

The verifier enforces an additional cross-field invariant: `actor === null ⇒ activeOrg === null` (rejected as `invalid: "actor_org_mismatch"`).

§5.3 App-side accessors

`createPlatformStartApp` (in `@greenroom/kit/react-start`) gains a third accessor:

```
// Existing
platform.getEnvelope(headers): EnvelopeData | null
platform.getSession(headers): PlatformSession | null
// New
platform.getActiveOrg(headers): EnvelopeActiveOrg | null
```

`getActiveOrg` is equivalent to `(await platform.getEnvelope(headers)).activeOrg ?? null` ; it's exposed as its own method so route loaders and server functions can write `const org = await platform.getActiveOrg(headers); if (!org) ...` without unwrapping `null` actor.

§5.4 Write-path verification

Reads trust the envelope. Writes that mutate org-scoped data (creating an org room, updating theme, inviting a member) **re-verify** through guestlist. Pattern from `docs/chat-spec.md` §1.2 generalizes:

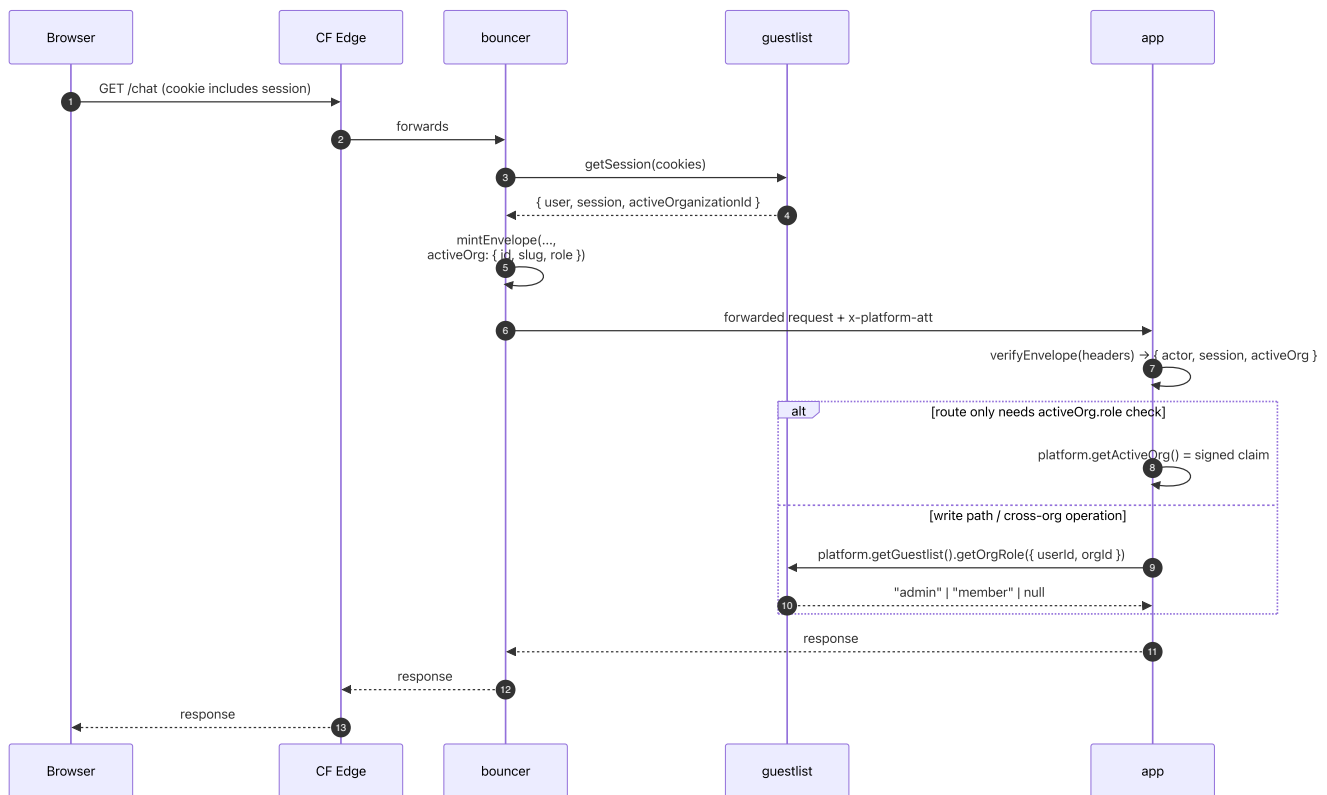
```

async function createOrgRoom({ orgId, title }, ctx) {
  const env = await platform.getEnvelope(ctx.headers);
  if (!env?.actor) throw new Error("unauthenticated");
  if (env.activeOrg?.id !== orgId) {
    // User is acting on an org that isn't their active one. Allowed -
    // we just don't trust the envelope's role for this org. Re-verify.
  }
  const role = await platform.getGuestlist().getOrgRole({
    userId: env.actor.id,
    orgId,
  });
  if (role !== "admin" && role !== "owner") throw new Error("forbidden");
  // ... proceed
}

```

In the common case (user is acting on their active org), the envelope's `activeOrg.role` is enough and the guestlist call is skipped. For admin-on-behalf-of and operator paths, the explicit `getOrgRole` check holds.

§5.5 Updated request lifecycle



The only new wire hop is guestlist computing `activeOrganizationId` on `getSession`, which is already in BA's session row — zero extra DB queries per envelope mint.

§6 — Guestlist API surface

There are **two distinct API surfaces** apps consume, with a clean decision rule for which to use. Confusing them is the easy mistake. This section pins down what BA's plugin already gives us vs. what we build.

§6.1 Surface A — auto-mounted by the BA org plugin

The moment we add `organization()` to the BA plugin list (O-0), the plugin **automatically mounts a full set of routes** under `/api/auth/organization/*`. We write zero lines of route code for these. Identity / chat / etc. consume them via the auth client returned by `createGuestlistClient` (the existing pattern, `services/guestlist/src/client/guestlist.ts`).

BA-mounted endpoint	Auth client call	Who can use it (default roles)
POST /api/auth/organization/create	<code>auth.organization.create({...})</code>	Any authenticated user (subject to <code>allowUserToCreateOrganization</code>)
GET /api/auth/organization/list	<code>auth.organization.list()</code>	Any authenticated user (returns only their memberships)
GET /api/auth/organization/get-full-organization	<code>auth.organization.getFullOrganization({...})</code>	Members of the target org
POST /api/auth/organization/update	<code>auth.organization.update({ data: { name?, slug?, logo?, metadata? } })</code>	Owners/admins of the target org
POST /api/auth/organization/delete	<code>auth.organization.delete({...})</code>	Owners
GET /api/auth/organization/check-slug	<code>auth.organization.checkSlug({ slug })</code>	Any authenticated user
POST /api/auth/organization/set-active	<code>auth.organization.setActive({...})</code>	Member of the target org
GET /api/auth/organization/list-members	<code>auth.organization.listMembers({...})</code>	Members of the target org
POST /api/auth/organization/add-member	<code>auth.organization.addMember({...})</code>	Owners/admins (with session headers); operator path is §6.2
POST /api/auth/organization/remove-member	<code>auth.organization.removeMember({...})</code>	Owners/admins (with session headers); operator path is §6.2
POST /api/auth/organization/update-member-role	<code>auth.organization.updateMemberRole({...})</code>	Owners/admins (with session headers); operator path is §6.2
POST /api/auth/organization/leave	<code>auth.organization.leave({...})</code>	Any member (of their own membership)
POST /api/auth/organization/get-active-member	<code>auth.organization.getActiveMember()</code>	Any authenticated user with an active org
POST /api/auth/organization/invite-member	<code>auth.organization.inviteMember({...})</code>	Owners/admins of the target org
POST /api/auth/organization/accept-invitation	<code>auth.organization.acceptInvitation({...})</code>	The invitee
POST /api/auth/organization/reject-invitation	<code>auth.organization.rejectInvitation({...})</code>	The invitee
POST /api/auth/organization/cancel-	<code>auth.organization.cancelInvitation({...})</code>	Owners/admins of the org that sent it

BA-mounted endpoint	Auth client call	Who can use it (default roles)
invitation		
GET /api/auth/organization/get-invitation	auth.organization.getInvitation({...})	The invitee (matched by email)
GET /api/auth/organization/list-invitations	auth.organization.listInvitations({...})	Members of the target org
GET /api/auth/organization/list-user-invitations	auth.organization.listUserInvitations()	The authenticated user (returns invites sent to them)
POST /api/auth/organization/has-permission	auth.organization.hasPermission({ permissions })	Any authenticated user

Key property of all of these: authority is derived from the session cookies on the request. The plugin checks "is the caller a member of the target org with the right role?" on every call. Operators **are not automatically allowed** through this surface unless they happen to also be members of the target org with the right role — which generally they're not.

§6.2 Surface B — custom operator routes we build

Operators (platform admins, `user.role === "admin"`) need to manage *any* org regardless of membership. BA's plugin doesn't expose a "platform-admin bypass" — it's strictly role-based on org membership. So we add a small set of custom Elysia routes on guestlist, all under `/admin/orgs/*`, all guarded by the existing `admin: true` Elysia macro (`services/guestlist/src/index.ts:159`), each one **calling the BA server API without session headers** (and passing `userId` / `organizationId` explicitly).

Custom route	Internal call	Added in CP
GET /admin/orgs	direct D1 select (BA has no "list all orgs" since it always scopes to caller's memberships)	O-3
POST /admin/orgs/create	auth.api.createOrganization({ body: { ..., userId }, /* no headers */ })	O-1
GET /admin/orgs/:id	auth.api.getFullOrganization({ query: { organizationId }, /* no headers */ }) if BA allows; else direct D1 select	O-5
POST /admin/orgs/:id/members	auth.api.addMember({ body: { userId, organizationId, role }, /* no headers */ })	O-6
POST /admin/orgs/:id/members/:userId/update-role	auth.api.updateMemberRole(...) — passing role + member id; may need a temporary impersonation header if BA's server-side method requires session context (TBD at implementation; fallback is a direct D1 update with beforeUpdateMemberRole hook invoked manually)	O-7
POST /admin/orgs/:id/members/:userId/remove	auth.api.removeMember(...) (same caveat as update-role)	O-7
POST /admin/orgs/:id/invitations	auth.api.createInvitation(...) if it accepts the no-headers + caller-override pattern; else a direct insert into the invitation table followed by manually calling the configured sendInvitationEmail callback	O-6
POST /admin/orgs/:id/invitations/:invitationId/cancel	auth.api.cancelInvitation(...) (same caveat)	O-7
GET /admin/users/search?email=	direct D1 select on user.email (not BA's concern — user lookup for the org-admin autocomplete)	O-4

Implementation honesty — empirically verified. Reading node_modules/better-

auth/dist/plugins/organization/routes/*.mjs and exercising each endpoint from the operator routes (landed in commit 139ca18):

- **No-headers + userId works** for createOrganization and addMember. BA's handlers call getSessionFromCtx(ctx).catch(() => null) and read body.userId / body.organizationId. Operator routes O-1 and O-6's add-member path use these directly.
- **No-headers refused** for updateMemberRole, removeMember, createInvitation, cancelInvitation — these declare requireHeaders: true (crud-members.mjs:208, :110; crud-invites.mjs:29, :362). Operator routes O-7's role-update + remove paths and O-6's invitation path fall back to **direct Drizzle writes**.

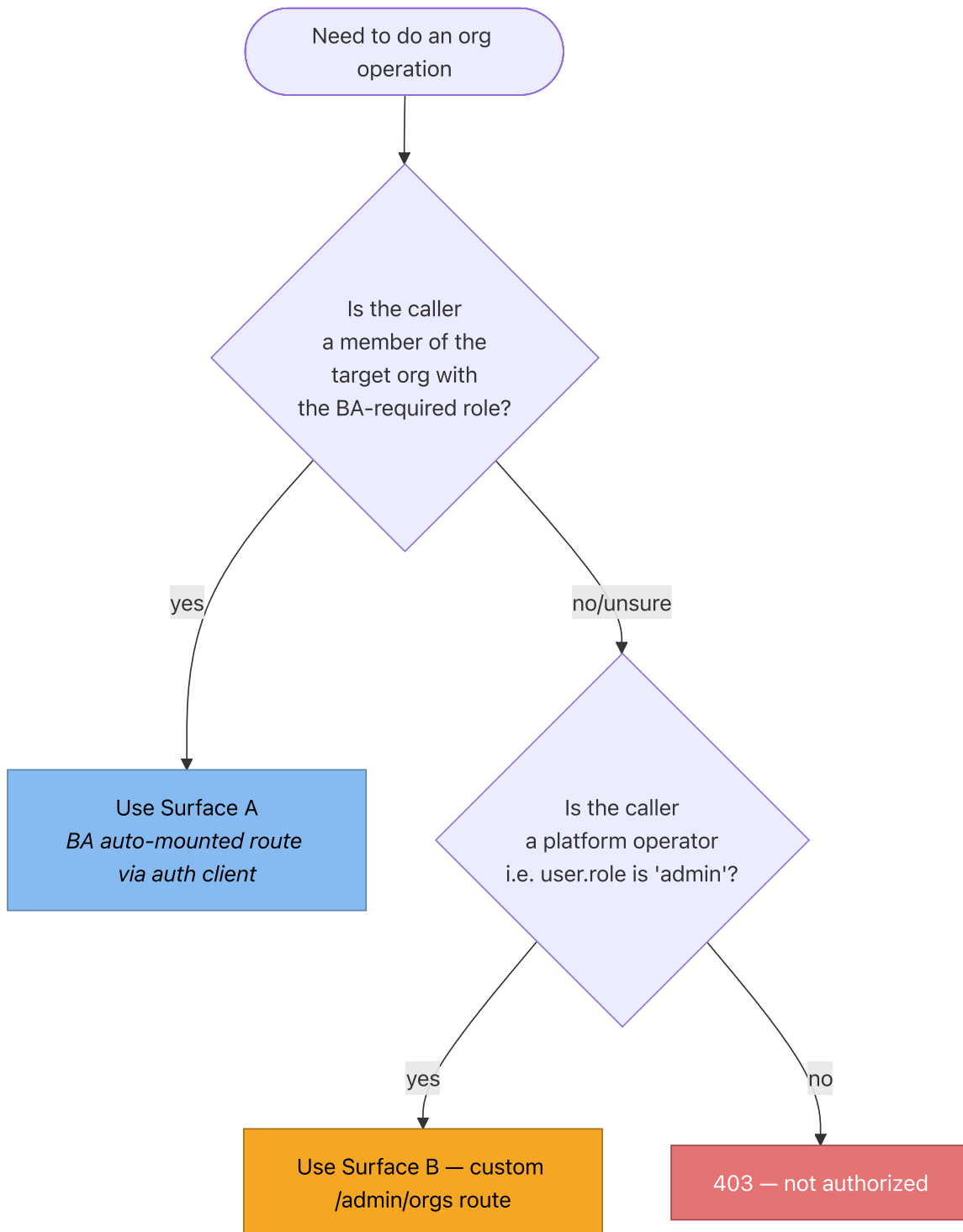
Implications of the Drizzle fallback:

- beforeUpdateMemberRole / afterUpdateMemberRole / beforeRemoveMember / afterRemoveMember / beforeCreateInvitation / afterCreateInvitation hooks defined on the org plugin config **do not fire** on operator paths. Brand-admin (Surface A) flows still hit BA's handler so hooks fire there as normal.
- The operator-issued invitation route does **not** trigger sendInvitationEmail (which only fires from BA's handler). The v1 operator UI returns the invitation id and surfaces a copyable accept URL; operator copies/pastes it. When a customer needs full email delivery on operator-issued invites, the move is to call promoter directly from the operator route handler — not to coerce BA into accepting no-headers.

If hook side-effects (audit logging, registration of derived data) ever need to fire on operator paths, the right move is to call into the same hook implementations directly from the operator route.

The operator UI in identity (`/admin/orgs/*` routes from O-3..O-7) calls these via TSS server functions which hit the custom guestlist routes through the service binding.

§6.3 Decision rule — which surface to call



The two surfaces never overlap. A brand admin configuring their own org always goes through Surface A. An operator configuring a customer's org always goes through Surface B. The same operation (e.g. "add a member") has two distinct call sites depending on which side the caller is on:

- Brand admin alice (Acme owner) adds bob to Acme → `authClient.organization.addMember({ userId: bob, role: "member" })` → BA route → BA checks alice is owner → done. (Surface A.)
- Operator carol adds bob to Acme without being an Acme member → identity TSS server fn → guestlist POST `/admin/orgs/<acme>/members` → guestlist's route handler calls `auth.api.addMember({ body: {...}, /* no headers */ })` → done. (Surface B.)

§6.4 Sugar helpers on `createGuestlistClient`

The chat-spec (P1) requires `getOrgRole` as a typed wrapper. We add a small helper set on the client returned by `createGuestlistClient`:

```
// services/guestlist/src/client/guestlist.ts (target – added to the returned object)
return {
  api,
  auth,
  async getSession(): Promise<PlatformSession | null> {
    /* existing */
  },

  /**
   * Returns the caller's role in the given org, or `null` if they're not
   * a member. Authoritative: hits guestlist, never trusts envelope claim.
   * Used by app write-paths that need to verify admin authority before
   * mutating org-scoped data.
   */
  async getOrgRole(args: {
    userId: string;
    orgId: string;
  }): Promise<"owner" | "admin" | "member" | null> {
    const res = await auth.organization.listMembers({
      organizationId: args.orgId,
      filterField: "userId",
      filterOperator: "eq",
      filterValue: args.userId,
      limit: 1,
    });
    return res.data?.members[0]?.role ?? null;
  },

  /**
   * Fetches an org's theme metadata + logo, formatted for app consumption.
   * Cached at guestlist with a 60s TTL (theme changes are rare); apps
   * can re-render on theme update via a webhook (future, §11.4).
   */
  async getOrgTheme(args: { orgId: string }): Promise<TenantTheme | null> {
    const res = await auth.organization.getFullOrganization({
      organizationId: args.orgId,
    });
    return res.data?.organization.metadata?.theme ?? null;
  },

  /**
   * Server-side permission check. Wraps BA's `hasPermission` with the
   * caller's active org. Apps that need RBAC enforcement against the
   * platform's statement set call this.
   */
  async hasOrgPermission(args: {
    permissions: Record<string, string[]>;
    orgId?: string;
  }): Promise<boolean> {
    const res = await auth.organization.hasPermission({
      permissions: args.permissions,
      ...(args.orgId && { organizationId: args.orgId }),
    });
    return res.data?.success === true;
  },
};
```

§6.5 Static vs dynamic access control

We start with **static roles** (the three built-ins + a small custom statement set per §4.1). Dynamic AC — letting org admins define their own roles at runtime — is a feature we'll ship when a customer asks. Enabling it is one flag and one migration (`organizationRole` table), so the deferral is genuinely cheap.

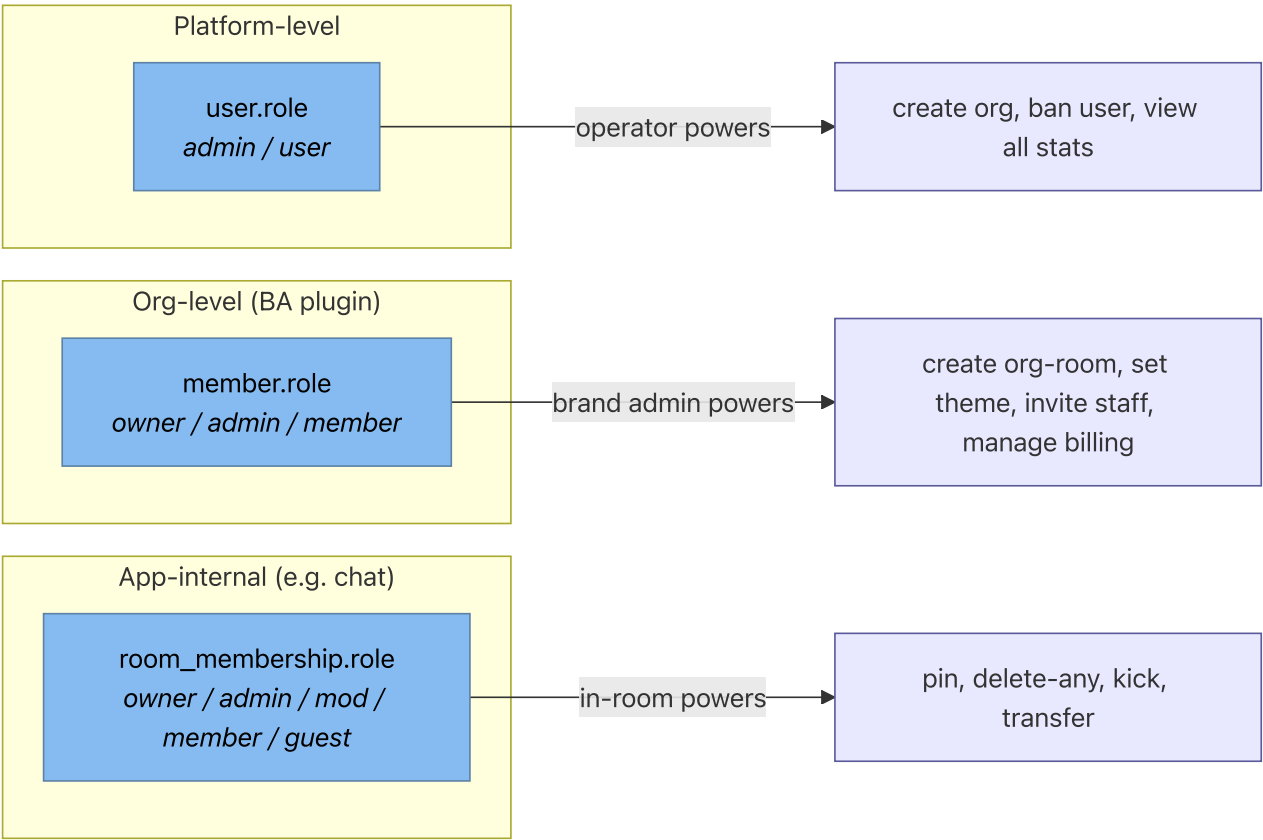
§6.6 Teams

The plugin's teams sub-feature is **off** in v1. Adding teams later is the same drill as dynamic AC: flag flip + migration (`team` + `teamMember` tables + `activeTeamId` on session). No design decisions blocked by waiting.

The use case for teams in our context is brand sub-orgs (Acme retail vs Acme wholesale). When that comes up, we enable teams and update §2.1's domain model.

§7 — Authorization model

Three role-bearing surfaces, each with a separate concern:



§7.1 Platform role

Stored on `user.role` (`admin` | `user`). Driven by BA's `admin` plugin, which is already wired. Platform admins (`role: "admin"`) are greenroom staff. They can do anything an org admin can do, on any org, plus operator-only things (create orgs, see all stats, ban users globally).

Apps check `envelope.actor.role === "admin"` for operator-only paths — this is unchanged from today.

§7.2 Org role

Stored on `member.role` . Driven by the BA org plugin (§4.1). Three values: `owner` , `admin` , `member` . Custom statements:

Statement	Role granted by default
<code>organization:update</code> , <code>organization:delete</code>	owner
<code>member:create</code> , <code>member:update</code> , <code>member:delete</code>	owner, admin
<code>invitation:create</code> , <code>invitation:cancel</code>	owner, admin
<code>theme:update</code>	owner, admin
<code>billing:read</code>	owner, admin
<code>billing:update</code>	owner
<code>scim:read</code> , <code>scim:configure</code>	owner

App code checks org role via `platform.getActiveOrg()` (envelope claim) for reads and `platform.getGuestlist().getOrgRole()` for writes (§5.4).

§7.3 App-internal role

App-specific. Chat's per-room roles (`owner` / `admin` / `mod` / `member` / `guest`) live in chat's D1, are managed by chat code, and don't propagate back to guestlist. Future apps (transfers, inventory, etc.) define their own internal role models the same way.

The bridge between org role and app role is **bootstrap on first use**: when a user with org-admin role connects to an org-owned chat room, chat auto-promotes them to room-admin (per `docs/chat-spec.md` §1.3). Same pattern applies elsewhere: org admins get app-level admin powers on first touch of org-owned resources, then those app roles are persisted in the app's own DB.

§7.4 The trust boundary table

Question	Where to ask	Trust level
Who is the user?	envelope <code>actor.id</code>	Signed, 30s TTL, sufficient
Is the user a platform admin?	envelope <code>actor.role === "admin"</code>	Signed, 30s TTL, sufficient
What's the active org id?	envelope <code>activeOrg.id</code>	Signed, 30s TTL, sufficient
Is the user an admin of their active org ?	envelope <code>activeOrg.role === "admin"</code>	Signed, 30s TTL, sufficient for reads
Is the user an admin of org X (not their active)?	guestlist <code>getOrgRole({ userId, orgId: X })</code>	Authoritative, required for writes
Can the user perform action Y on org X?	guestlist <code>hasOrgPermission({ permissions: ..., orgId })</code>	Authoritative, required for writes

Apps follow the same rule chat does: **envelope for reads, guestlist for writes**. The envelope's 30s window is acceptable for "show the admin menu" but not for "delete this room".

§8 — White-label

The platform's design system (`packages/design/`) is already CSS-variable driven and theme-mode aware (`:root + [data-theme="dark"]`) + system preference, per the inventory below). Per-org theming is a **third layer**: runtime overrides injected after the design-system defaults but before component styles resolve.

§8.1 Token surface — the tenant contract

The design system has **two layers** of CSS variables, and tenants only touch one of them.

§8.1.1 Internal palette (NOT tenant-overridable)

`packages/design/src/tokens/colors.ts` defines a private palette with project-specific names (`--color-glyph` , `--color-blood` , `--color-verdigris` , `--color-ochre` , `--color-slate` , plus structural tokens like `--color-bg` , `--color-surface` , `--color-text`). These names were inherited from the platform-template's prior design system ("glyph") and are an **implementation detail of greenroom's own default theme**. They can be renamed, replaced, or thrown out wholesale in a future redesign; no tenant code or theme override should reference them.

The platform's own components don't reference these directly either — they consume the shadcn semantic layer below.

§8.1.2 Shadcn semantic layer (the tenant contract)

`packages/design/src/theme.css` maps the standard shadcn semantic tokens onto the internal palette (current mapping is informational only — the RHS can change):

```
@theme inline {
  --color-background:      var(--color-bg);
  --color-foreground:      var(--color-text);
  --color-card:            var(--color-surface-raised);
  --color-card-foreground: var(--color-text);
  --color-popover:         var(--color-surface-raised);
  --color-popover-foreground: var(--color-text);
  --color-primary:         var(--color-glyph);
  --color-primary-foreground: var(--color-text-on-accent);
  --color-secondary:       var(--color-surface);
  --color-secondary-foreground: var(--color-text);
  --color-muted:           var(--color-surface-sunken);
  --color-muted-foreground: var(--color-text-secondary);
  --color-accent:          var(--color-surface);
  --color-accent-foreground: var(--color-text);
  --color-destructive:     var(--color-blood);
  --color-border:          var(--color-border);
  --color-input:           var(--color-border-strong);
  --color-ring:            var(--color-glyph);
  --color-chart-{1..5}:    /* mapped to internal palette */
  --color-sidebar-*:       /* mapped to internal palette */
}
```

These shadcn token names are the **stable, universal contract** that multi-tenant surfaces use. Every shadcn-generated component in `packages/ui` already consumes them. A tenant theme overrides the shadcn tokens — not the internal palette — and the override propagates through every component without touching greenroom's own theme code.

§8.1.3 The tenant-overridable allowlist

A tenant theme may set values for **these keys only**. Everything else is rejected at the theme-editor server fn (validation) and stripped at `<0rgThemeStyle>` render time (defense in depth, MT5):

Group	Keys
Surfaces	<code>--color-background</code> , <code>--color-foreground</code>
Cards	<code>--color-card</code> , <code>--color-card-foreground</code>
Popovers	<code>--color-popover</code> , <code>--color-popover-foreground</code>
Primary	<code>--color-primary</code> , <code>--color-primary-foreground</code>
Secondary	<code>--color-secondary</code> , <code>--color-secondary-foreground</code>
Muted	<code>--color-muted</code> , <code>--color-muted-foreground</code>
Accent	<code>--color-accent</code> , <code>--color-accent-foreground</code>
Destructive	<code>--color-destructive</code>
Borders / focus	<code>--color-border</code> , <code>--color-input</code> , <code>--color-ring</code>
Charts	<code>--color-chart-1</code> ... <code>--color-chart-5</code>
Sidebar	<code>--color-sidebar</code> , <code>--color-sidebar-foreground</code> , <code>--color-sidebar-primary</code> , <code>--color-sidebar-primary-foreground</code> , <code>--color-sidebar-accent</code> , <code>--color-sidebar-accent-foreground</code> , <code>--color-sidebar-border</code> , <code>--color-sidebar-ring</code>
Radius (single value)	<code>--radius</code> (shadcn's base radius; the rest derive)

The allowlist is exported from `@greenroom/design` as a constant (`SHADCN_TENANT_TOKEN_ALLOWLIST`) so the validator, the editor UI, and the `<OrgThemeStyle>` sanitizer share one source of truth.

Out of scope for v1: per-org typography overrides (`--font-*` , `--text-*`), per-org spacing/breakpoints/shadows. If a customer needs a custom display font, we add `--font-sans` and `--font-heading` to the allowlist with a font-file upload path — separate checkpoint, not shipped initially. Per-org radii feel possible but underdemanded; add when asked.

§8.2 Override injection

Org theme reaches the browser as a small inline `<style>` block in the document `<head>` , injected by the app's root layout. The pattern:

```
// apps/<app>/src/routes/__root.tsx (target – adds OrgThemeStyle)
import { OrgThemeStyle } from "@greenroom/ui/components/org-theme-style";

export const Route = createRootRouteWithContext()({
  beforeLoad: async ({ context }) => {
    const activeOrg = await context.platform.getActiveOrg(getRequestHeaders());
    const theme = activeOrg
      ? await context.platform.getGuestlist().getOrgTheme({ orgId: activeOrg.id })
      : null;
    return { activeOrg, theme };
  },
  component: ({ context }) => (
    <RootDocument>
      <ThemeInitScript />
      <OrgThemeStyle theme={context.theme} />
      <Outlet />
    </RootDocument>
  ),
});
```

`OrgThemeStyle` renders something like:

```

<style id="org-theme">
  :root {
    --color-primary: #7c3aed;
    --color-primary-foreground: #ffffff;
    --color-ring: #7c3aed;
    --color-destructive: #ef4444;
    /* ... only allowlisted shadcn keys the tenant overrode */
  }
  [data-theme="dark"] {
    --color-primary: #a78bfa;
    --color-primary-foreground: #0a0a0a;
    --color-ring: #a78bfa;
    /* ... */
  }
</style>

```

Order matters. The injection point is **after** `packages/design` 's imported CSS (which defines defaults) and **before** any component CSS. The existing `THEME_INIT_SCRIPT` (which sets `data-theme` from `localStorage`) runs first to fix the mode; `OrgThemeStyle` then overlays org-specific values for whichever mode wins.

For the operator-context view (no active org), `OrgThemeStyle` renders nothing — platform defaults hold.

§8.3 Logo and wordmark

The shared `Logo` component (`packages/ui/src/components/ui/logo/logo.tsx`) currently pulls `platformConfig.brand.name` at render time. Target: accept an optional override from a context provider, fall back to platform default:

```

// packages/ui/src/components/ui/logo/logo.tsx (target – context-aware)
import { useTenantBrand } from "@greenroom/ui/tenant-brand";

export function Logo(props: LogoProps) {
  const tenant = useTenantBrand();
  const wordmark = tenant?.wordmark ?? platformConfig.brand.name;
  const logoIcon = tenant?.logoRefId ? (
    <img src={`/u/logo/${tenant.logoRefId}`} alt={wordmark} />
  ) : (
    <LogoIcon {...props} />
  );
  // ...
}

```

The `TenantBrand` context provider lives in `packages/ui/src/tenant-brand.tsx` and is populated by each app's root layout from the active-org theme. Apps that don't want per-org branding just don't wrap in the provider.

§8.4 OG image rendering

Identity's OG image route (`apps/identity/og/_brand.tsx`) is the trickier case — Satori doesn't run the browser's CSS pipeline, so the OG renderer can't read CSS vars. Today it inlines literal HSL values from the design tokens. Target: when rendering an OG image for an org-scoped surface (e.g. an invite landing page), accept an `orgId` query param, look up the org's theme via guestlist, and inline the tenant's HSL values:

```
// apps/identity/og/_brand.tsx (target — accepts org context)
export function OgBrand({ subtitle, orgId }: { subtitle?: string; orgId?: string }) {
  const tenantTheme = orgId ? useOrgThemeForOg(orgId) : null;
  const colors = tenantTheme ? resolveOgColorsFromTheme(tenantTheme) : DEFAULT_OG_COLORS;
  return (
    <div style={{ background: colors.bg, color: colors.text }}>
      <LogoIcon colorScheme="light" colors={colors.glyphColors} />
      <h1 style={{ color: colors.text }}>{tenantTheme?.wordmark ?? platformConfig.brand.name}</h1>
      {subtitle && <p style={{ color: colors.subtitle }}>{subtitle}</p>}
    </div>
  );
}
```

OG fetches happen out-of-request (Slack unfurl, Twitter card), so we can't rely on cookies or envelope. The `orgId` is the URL param; the OG renderer authenticates to guestlist via the existing service binding (no user cookie needed for public org metadata).

§8.5 Bouncer and the "asset broker" pattern

`packages/ui` and `identity` already use the `/u/avatar/:refId` pattern for user avatars: a public, cacheable redirect through guestlist into roadie's signed-URL flow. Org logos use the same shape at `/u/logo/:refId`:

- App renders ``.
- That URL resolves on the org's subdomain (or path-scoped), bouncer proxies `/u/*` to guestlist (same routing as avatars, per `docs/ARCHITECTURE.md §3.2 Invariants`).
- Guestlist returns a 302 to a short-lived presigned R2 URL from roadie.
- Browser caches the redirect for 5 min (shorter than the presign TTL).

This means: **no new infra**. Org logos reuse the avatar plumbing end to end. Roadie's `application` field gets a new resource type: `{ app: "guestlist", resourceType: "org-logo", resourceId: orgId }`.

§8.6 Theme editor

Identity gains a route `/orgs/:slug/theme` with three views:

1. **Color editor** — token-name list, color pickers, live preview pane. Saves to `organization.metadata.theme.{light,dark}`.
2. **Logo uploader** — drag-drop SVG/PNG → registers in roadie via the existing avatar three-step flow (`register` → `PUT` → `confirm`) wired to `/api/org-logo/*` (new endpoints, mirror `/api/avatar/*`).
3. **Wordmark editor** — text input, saves to `organization.metadata.theme.wordmark`.

Role required: `theme:update` (granted to org admins and owners per §4.1). Permission check via `auth.organization.hasPermission`.

§9 — Dev/prod parity

The platform's dev story (`docs/ARCHITECTURE.md §4.5`) is "apps run alone, service-bind to a local guestlist, envelope absent in dev". This holds for multi-tenancy with one addition.

§9.1 Seed orgs

`scripts/bootstrap.ts` (target — extend the existing bootstrap) seeds a couple of orgs in the local D1 so dev sessions have somewhere to live:

```
// scripts/seed-orgs.ts (target)
await db.insert(organization).values([
  {
    id: "org_dev_acme",
    slug: "acme",
    name: "Acme Cannabis",
    metadata: {
      theme: {
        schemaVersion: 1,
        light: {
          "--color-primary": "#7c3aed",
          "--color-primary-foreground": "ffffff",
          "--color-ring": "#7c3aed",
        },
        dark: {
          "--color-primary": "#a78bfa",
          "--color-primary-foreground": "#0a0a0a",
          "--color-ring": "#a78bfa",
        },
        wordmark: "Acme",
      },
    },
    createdAt: Date.now(),
  },
  {
    id: "org_dev_beta",
    slug: "beta",
    name: "Beta Co",
    metadata: { theme: { wordmark: "Beta" } },
    createdAt: Date.now(),
  },
]);
// Plus a default member row tying any seeded user to org_dev_acme as owner.
```

The seed runs as part of `bun run bootstrap`, gated on `ENVIRONMENT === "development"`.

§9.2 Multi-org dev flow

A dev signs in as the seeded user, picks their active org via the identity switcher, and every app's chrome reflects the seeded theme. Switching to the other org demonstrates the live re-theme.

Apps running standalone (no bouncer, no envelope — the dev-direct topology from `docs/ARCHITECTURE.md §4.5`) get their active org from `getSession()` instead of the envelope. The kit's `getActiveOrg(headers)` already encapsulates this fallback: in dev it calls `getSession()`, in prod it reads from the envelope. Apps never branch on `ENVIRONMENT`.

§9.3 Test orgs

E2E tests need stable org fixtures. Pattern: create per-test orgs with ULID slugs, seed memberships, tear down. Helper lives in `packages/test-utils` (to be created when the third test file needs it).

§10 — Domain strategy

§2.5 outlined the path-per-org vs subdomain-per-org tradeoff. Here's how each is wired.

§10.1 Path-per-org (v1 default)

```
greenroom.app/o/acme/chat      → bouncer → chat app
greenroom.app/o/acme/identity  → bouncer → identity app
greenroom.app/o/acme/orgs/theme → bouncer → identity app (theme editor)
```

Bouncer's `ROUTES` config has one rule per app, parameterized by the `/o:slug` prefix:

```
// services/bouncer/wrangler.template.jsonc (target)
"vars": {
  "ROUTES": [
    { "match": "{{BASE_DOMAIN}}/o:slug/chat/*",    "binding": "CHAT",    "mode": "passthrough" },
    { "match": "{{BASE_DOMAIN}}/o:slug/*",         "binding": "IDENTITY", "mode": "passthrough" },
    { "match": "{{BASE_DOMAIN}}/*",               "binding": "IDENTITY", "mode": "passthrough", "auth": {
      "require": "operator" } }
  ]
}
```

Bouncer **does not parse the slug for routing decisions** beyond picking the binding. The slug → orgId resolution happens inside `guestlist` when the app asks `getActiveOrg`. Bouncer just forwards the path; the app reads it.

The active-org resolution path:

1. User signs in. Default `activeOrganizationId` is set per §4.2.
2. User visits `greenroom.app/o/acme/chat`. App's root loader sees the URL slug.
3. App reconciles: if `envelope.activeOrg.slug !== "acme"`, app calls `auth.organization.setActive({ organizationSlug: "acme" })` (only if user is a member), which updates the session row. Next request's envelope reflects the new active org.
4. If user isn't a member of `acme`, app shows a "you don't have access" page.

The slug-in-URL is a **UX hint**, not the authority. The authority is the session row, signed forward via the envelope.

§10.2 Subdomain-per-org

```
acme.greenroom.app      → bouncer → identity (or chat with subroute)
acme.greenroom.app/chat → bouncer → chat
acme.greenroom.app/settings/theme → bouncer → identity
```

Bouncer `ROUTES` use host-match instead of path-prefix-match. The host's first label is the slug; bouncer doesn't care, it just routes to the right binding. App-side, the same `setActive` reconciliation happens against the host instead of the path.

Cookies must scope to `*.greenroom.app` (already the case via `AUTH_DOMAIN`'s leading-dot convention) so the session shares across `acme.greenroom.app` and `beta.greenroom.app` without re-auth.

§10.3 Custom domains

`chat.acme.com` → bouncer → chat (with `activeOrg = Acme`). Wiring:

1. Tenant adds a CNAME from `chat.acme.com` to the platform's CF Custom Domain hostname.
2. Operator (or self-serve UI, future) registers `chat.acme.com` as a CF Custom Domain on bouncer's Worker.
3. Operator (or self-serve UI) creates a `tenant_domain` row mapping `chat.acme.com` → `orgId=acme`.
4. Bouncer's session-refresh stage, when minting the envelope, looks up the host in the `tenant_domain` table and stamps `activeOrg` from there instead of from the session row. (The cookie still authenticates the user; the domain pins the active org.)

This is the cleanest UX (no `/o/<slug>` in the URL, no `acme.greenroom.app` "platform-y" feel) and the way enterprise customers will want to deploy. Build the table and lookup in v1 even if we don't ship the operator UI immediately — the bouncer code is small and isolation matters once it's live.

§11 — SCIM (deferred)

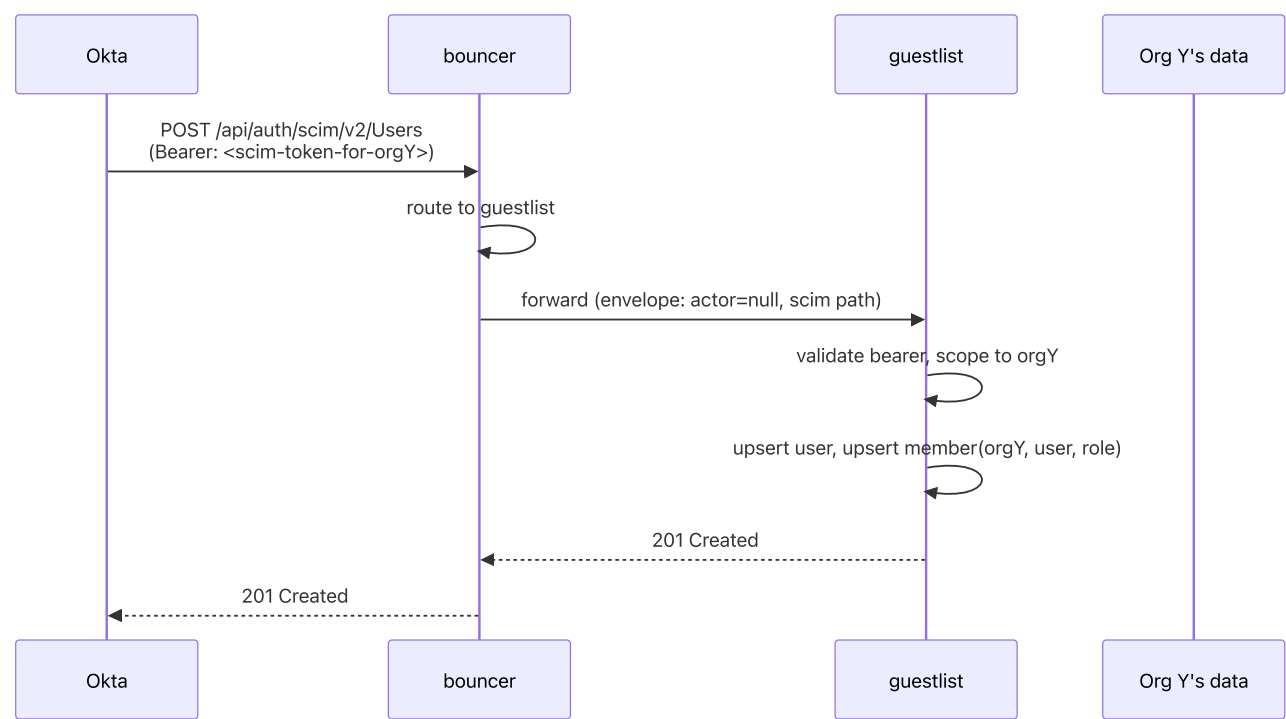
Enterprise customers will provision and deprovision their staff from Okta/Entra/Azure AD via SCIM 2.0. Our take: a separate plugin in `@greenroom/auth`'s plugin list, conditional on a per-org flag.

§11.1 Surface

POST	/api/auth/scim/v2/Users	(provision user, attach to org)
GET	/api/auth/scim/v2/Users/:id	
PATCH	/api/auth/scim/v2/Users/:id	(suspend, role change)
DELETE	/api/auth/scim/v2/Users/:id	(deprovision)
POST	/api/auth/scim/v2/Groups	(mirror BA roles? deferred)
GET	/api/auth/scim/v2/ServiceProviderConfig	
GET	/api/auth/scim/v2/Schemas	
GET	/api/auth/scim/v2/ResourceTypes	

Authentication: per-org SCIM bearer token (BA `apikey` plugin already handles the token storage — we add a `scim` API-key scope). Each token is scoped to one org; SCIM operations only touch that org's members.

§11.2 Data flow



SCIM provisioning **does not invite via email**. It auto-creates the user (no password — they'll sign in via the brand's IdP through guestlist's `oauthProvider` or OIDC plugin), creates the member row directly, and returns the SCIM-canonical user representation.

§11.3 Identity binding

The provisioned user's email is the bridge. When the user signs in via the brand's SSO, BA's account-linking (already enabled, `packages/auth/src/server.ts:103`) attaches the SSO account to the existing user row. The org membership pre-exists; the user is "ready to go" the moment they sign in.

§11.4 Build plan

Phase out:

1. **O-SCIM-1**: BA plugin scaffold. Bearer auth via apikey, JSON shape compliance, single-user CRUD against the existing `user` and `member` tables.
2. **O-SCIM-2**: Group mapping (Okta groups → org roles), if a customer needs it.
3. **O-SCIM-3**: Bulk operations, filters, pagination — required for full SCIM 2.0 conformance, often the gating item for Okta App Gallery listings.

None of this lands before a customer asks. The data model already supports it (members + roles + apikeys); the gap is the SCIM-shape HTTP adapter.

§12 — Checkpoint plan

Built like `docs/chat-spec.md`: small, demoable steps. Each step is one PR-sized chunk. Each one ends in a **Verify** instruction the operator can actually run.

The first twelve checkpoints (O-0 ... O-11) are the manual-onboarding MVP. At O-9 the operator can sign in, provision a new org, invite a brand admin by email, and have them accept. At O-11 the invited admin can see their active org in the header switcher across all apps — no theme yet, no chat scoping yet, but the spine is real. O-12 onward adds chat scoping, theme, logo, and routing polish. The far-future items (subdomain, custom domains, SCIM, dynamic AC, teams, self-signup) live in Phase 7 §Deferred.

Implementation status — 2026-05-17

✅ **Landed on `worktree-organizations`** (manual-onboarding MVP): O-0, O-1, O-3, O-4, O-5, O-6, O-7, O-8, O-9. Full demo path works in local dev — sign in as admin, provision an org, add/invite members, have invitees accept. Backend has 24/24 guestlist org tests passing; all four migrations apply cleanly from a fresh D1.

🟡 **Deferred** (not blocking the demo):

- **O-2** (chat-side `getOrgRole` sugar) — wait until chat actually consumes it. Operator routes don't need it.
- **O-4 owner-role radio** — the spec mentioned an initial-role picker so the operator could create an org and assign the new user as `admin` rather than `owner`. v1 always assigns `owner`; the operator workaround is "create with my own userId, add the brand admin as admin, optionally promote/transfer later" (still one extra click).
- **Slug-availability live-check** in `/admin/orgs/new` — would need the BA `organizationClient()` plugin registered in `services/guestlist/src/client/plugins.ts`. Currently the form surfaces a server-side 409 on submit as an inline field error, which is fine.

■ **Pending** (next demo would unblock): O-10 (envelope `activeOrg`), O-11 (kit accessor + switcher), O-12 (chat consumes), O-13..O-16 (theme + logo + editor), O-17 (OG), O-18 (path-per-org bouncer routes).

■ **Far-deferred**: O-19..O-24 (subdomain, custom domains, SCIM, dynamic AC, teams, self-signup) — no near-term forcing function.

Phase 1 — Plumbing only (no UI). Unblocks chat-spec P1.

O-0 — Add BA org plugin to @greenroom/auth

Smallest possible PR: add `organization()` to `createPlatformAuth`'s plugin array. Just the three default roles, no custom AC yet, no hooks, `allowUserToCreateOrganization: false`. No `sendInvitationEmail` callback yet — wire to a no-op stub that throws "TODO" so invitations fail loudly. **No identity UI changes.**

Generate the BA migration (`cd services/guestlist && bun run codegen`), review it, commit it, apply locally (`bun run migrate`).

🔧 **Files:** `packages/auth/src/server.ts`, `services/guestlist/drizzle/<timestamp>_organization.sql` (generated), `services/guestlist/src/schema.ts` (regenerated mirror — do **not** type `metadata` here yet; that lands in O-11 to keep this PR boring). 🕒 **Verify:** `bun run check clean`; D1 has `organization`, `member`, `invitation`, and the `session.active_organization_id` column; `bun run dev` boots all services cleanly; sign-in still works.

O-1 — Operator route in guestlist: `POST /admin/orgs/create`

Surface B (§6.2 — the first custom operator route). BA's auto-mounted `POST /api/auth/organization/create` only creates orgs owned by the calling user; for the "operator creates org owned by someone else" flow we wrap `auth.api.createOrganization` with no session headers + explicit `userId`. Add a single Elysia route guarded by the existing `admin: true` macro. Pure server work; no identity UI yet. The chat-spec P1 unblock can happen at this point if you stub a couple memberships via raw SQL — but realistically O-2 lands at the same time.

🔧 **Files:** `services/guestlist/src/index.ts`. 🕒 **Verify:** `curl -X POST` with the operator session cookie creates an org row owned by the specified `userId`. `select * from organization` in local D1 shows the row.

O-2 — Guestlist client sugar: `getOrgRole`

Single helper on the object returned by `createGuestlistClient`. Just enough to unblock chat's per-call brand-admin verification (chat-spec §1.2). Other helpers (`getOrgTheme`, `hasOrgPermission`) come later.

🔧 **Files:** `services/guestlist/src/client/guestlist.ts`. 🕒 **Verify:** from chat's server fn, with a manually-seeded (`userId=X`, `orgId=Y`, `role=admin`) member row, `platform.getGuestlist().getOrgRole({ userId: X, orgId: Y })` returns "admin".

At this point chat-spec P1 is unblocked. Chat-spec CP-15+ can build against `getOrgRole` even before identity has any org UI.

Phase 2 — Identity admin surface for operator-driven onboarding

O-3 — Identity `/admin/orgs` list page

Surface B (§6.2). BA's `auth.organization.list()` scopes to the caller's memberships; an operator needs to see *all* orgs whether or not they're a member. Adds `GET /admin/orgs` on guestlist (direct D1 select) and the identity admin-shell route. Operator-only. The existing `/admin/*` routes share an admin shell (currently hosts `/admin/sessions`, `/admin/api-keys`, `/admin/clients`). We add **Organizations** to that shell as a new top-level nav entry.

On screen:

- Nav entry **Organizations** in the admin sidebar, between **Users** (if present) and **API keys**.
- Page header: "Organizations" with a primary button **+ New organization** linking to `/admin/orgs/new`.
- Table with columns: **Slug** (monospaced, link to detail page), **Name**, **Members** (count), **Created** (relative time), **Owner** (display name of the `owner` -role member, or "—" if none).
- Default sort: created desc.
- Pagination: 50/page, server-side via BA's `listOrganizations` (or a direct admin Elysia route on guestlist returning the join with member count if BA's pagination isn't ergonomic enough).
- Empty state: "No organizations yet. Onboard your first brand →" pointing at `/admin/orgs/new`.

🔗 **Files:** `apps/identity/src/routes/admin.orgs.tsx`, `apps/identity/src/components/admin/admin-shell.tsx` (extend nav), `apps/identity/src/lib/org-admin.functions.ts` (server fn: `listOrgsForAdmin`). 🧐 **Verify:** operator sees the empty state on a fresh DB. After O-1 creates an org via curl with two members, the table shows it with **Members: 2** and the owner's display name.

O-4 ✅ — Identity `/admin/orgs/new` create form

Surface B for org creation (reuses O-1's route), **plus a new GET `/admin/users/search?email=` route** on guestlist for the owner autocomplete (user-lookup, not part of BA's plugin scope). The slug availability check uses BA's auto-mounted `auth.organization.checkSlug` (Surface A) since it doesn't require membership. Operator-only. Single-column form, full width of admin shell content area.

Fields:

- **Organization name** — text input, required, 2–60 chars. Pure human-readable string.
- **Slug** — text input, required, lowercase kebab-case (`/^[a-z0-9](?:[a-z0-9-]{1,38}[a-z0-9])?$/`). Below the input, a helper line shows the resulting URL form: `/o/<slug>/*` . Debounced live availability check via `auth.organization.checkSlug({ slug })` ; inline status: "available" / "taken" / "invalid". Auto-suggest from name on first keystroke (kebab-case the name, dedupe).
- **Owner email** — email input + async autocomplete against `user.email` via a new server fn `searchUsersByEmail` . Only matches users that already exist in the DB. Selecting an entry pins the `userId` .
- **Initial role for owner** — radio: `owner` (default) | `admin` . (If operator wants to keep ownership themselves for v1 demos, they pick `admin` here and add themselves as `owner` separately. Either is fine.)

On submit calls the `createOrgAsOperator` server fn from §4.4:

- Success → toast "Created " → navigate to `/admin/orgs/:id` .
- Slug collision → inline error on the slug field.
- Email not found → inline error on the owner-email field, with a link "Send them a sign-up invite instead" (which calls BA's `magicLink` / regular signup-link flow — pre-existing, just point at it).

🔗 **Files:** `apps/identity/src/routes/admin.orgs.new.tsx`, `apps/identity/src/lib/org-admin.functions.ts` (`createOrgAsOperator`, `searchUsersByEmail`), `services/guestlist/src/index.ts` (the underlying `POST /admin/orgs/create` from O-1 already exists; add `GET /admin/users/search?email=` for autocomplete). 🧐 **Verify:** as operator, sign up two test users out-of-band; create an org with one as `owner` ; the org appears in `/admin/orgs` ; the `member` row in D1 has `role: "owner"` and the right `userId` .

O-5 ✅ — Identity `/admin/orgs/:id` detail page (read-only)

Surface B. BA's `auth.organization.getFullOrganization` and `listInvitations` both require membership, so we add `GET /admin/orgs/:id` (operator-only) that internally calls them with no session headers (or falls back to a direct D1 join if BA refuses that path — see §6.2's implementation-honesty note). Operator-only. Lands after O-4 and from clicking a slug in O-3's table. Pure read view; mutation actions land in O-6 and O-7.

On screen:

- Page header: org **Name** with the **Slug** as a monospaced sub-label.
- Three card-style sections stacked vertically:
 1. **Overview** — created date, member count, plan (placeholder for v1 — always "trial"), theme presence indicator (a small swatch showing the org's `--color-primary` if set, "default" otherwise).
 2. **Members** — table: avatar + display name + email, role badge (`owner` / `admin` / `member`), joined date. Sort by role then name.
 3. **Pending invitations** — table: email, role, expires (relative time), invited by. Hidden entirely if list is empty.
- No buttons yet (actions land in O-6/O-7). The page is read-only.

🔗 **Files:** `apps/identity/src/routes/admin.orgs.$id.tsx`, `apps/identity/src/lib/org-admin.functions.ts` (`getOrgForAdmin({ orgId })` — wraps BA's `getFullOrganization` + `listInvitations`). 🧐 **Verify:** operator opens an org created in O-4; sees the owner listed in Members; sees an empty Pending invitations table.

O-6 — /admin/orgs/:id — add and invite members

Surface B. Two new operator routes: `POST /admin/orgs/:id/members` (direct add, wraps `auth.api.addMember` no-headers) and `POST /admin/orgs/:id/invitations` (operator-issued invitation — implementation per §6.2's note: BA's `createInvitation` no-headers if supported, else direct D1 insert + manual `sendInvitationEmail` invocation). Operator-only. Extends O-5 with **two buttons in the Members card header**:

- **+ Add member** — opens a modal with: email autocomplete (same as O-4, only matches existing users), role select (`member` / `admin` / `owner`). Calls a new server fn `addMemberAsOperator({ orgId, userId, role })` which hits a new guestlist route `POST /admin/orgs/:id/members` that calls `auth.api.addMember({ body: { userId, organizationId, role }, /* no headers */ })`. Success → modal closes, members table refreshes, toast "Added".
- **+ Invite by email** — opens a modal with: email input (free-form, no autocomplete; can target users who don't exist yet), role select. On submit calls `auth.organization.inviteMember(...)`. Until O-8 lands, this writes the invitation row but doesn't send email; the modal surfaces the raw invite link (`${IDENTITY_URL}/orgs/accept/<invitationId>`) so operator can copy/paste while we wait on the promoter wire-up. After O-8 the modal also says "Invitation email sent."

 **Files:** `apps/identity/src/routes/admin.orgs.$id.tsx` (extend), `apps/identity/src/components/admin/add-member-modal.tsx`, `apps/identity/src/components/admin/invite-member-modal.tsx`, `apps/identity/src/lib/org-admin.functions.ts` (extend), `services/guestlist/src/index.ts` (new `POST /admin/orgs/:id/members`).  **Verify:** add an existing user as `member` — they appear in the table immediately. Invite a non-existent email as `admin` — the invitation appears in Pending invitations, the modal shows a copyable accept link.

O-7 — /admin/orgs/:id — change role and remove member

Surface B. Three new operator routes: `POST /admin/orgs/:id/members/:userId/update-role`, `POST /admin/orgs/:id/members/:userId/remove`, `POST /admin/orgs/:id/invitations/:invitationId/cancel` — all wrapping the BA server API with no headers (or direct DB writes if BA refuses, per §6.2). Operator-only. Extends the Members table from O-6 with **per-row action menu** (`...`):

- **Change role** — submenu with `owner` / `admin` / `member`. Calls `auth.api.updateMemberRole({ body: { memberId, role }, /* no headers */ })` via a new operator route. Cannot demote the last owner (server-side check; client also disables the option). Optimistic update with rollback on error.
- **Remove from org** — confirmation modal ("Remove from ? They'll lose access immediately."). Calls a new operator server fn → `POST /admin/orgs/:id/members/:userId/remove` → `auth.api.removeMember(...)`. On success the row disappears.
- The Pending invitations table from O-5 gets its own action menu: **Resend invitation** (calls `inviteMember` with `resend: true`), **Cancel** (calls `cancelInvitation`).

Cannot remove yourself if you're the operator (a UI guard; operator auto-membership only matters if §4.4's "operator-as-owner" flow was used).

 **Files:** `apps/identity/src/routes/admin.orgs.$id.tsx` (extend), `apps/identity/src/components/admin/member-actions.tsx`, `apps/identity/src/lib/org-admin.functions.ts` (extend), `services/guestlist/src/index.ts` (new `POST /admin/orgs/:id/members/:userId/update-role` and `.../remove`).  **Verify:** change a member's role; reload — sticks. Try to demote the only `owner` — UI blocks; if you bypass via curl, server rejects. Remove a member; refresh — gone from the table and from `member` rows in D1.

O-8 — Promoter organization-invitation template + BA wire-up

Add the `organization-invitation` kind to promoter's payload union and template set (matches existing email kinds). Wire guestlist's `sendInvitationEmail` callback to call `env.PROMOTER.send(...)`. Replace the O-0 throwing stub with this. The O-6 invite modal stops surfacing the raw link (or keeps it as a "Copy link" fallback for envs without `RESEND_API_KEY`).

 **Files:** `services/promoter/src/templates.ts`, `services/promoter/src/index.ts`, `services/guestlist/src/auth-config.ts` (or `services/guestlist/src/index.ts` depending on where `sendInvitationEmail` lands cleanest), `apps/identity/src/components/admin/invite-member-modal.tsx` (replace raw-link with "Invitation sent" success state). 

Verify: invite a user by email via O-6's modal; they receive the invitation email via Resend (or the local Resend stub); URL points to `/orgs/accept/:invitationId`; the invitation in D1 shows `status: "pending"`.

O-9 — Identity `/orgs/accept/:invitationId` landing

Surface A. The invitee themselves is the caller, so we use BA's auto-mounted `auth.organization.getInvitation / acceptInvitation / rejectInvitation` directly via the auth client. No new guestlist routes. Public route. The user lands here from an invitation email.

UI flow:

- **Not signed in:** redirect to `/sign-in?returnTo=/orgs/accept/:id`. Sign-in / sign-up both work (BA's `requireEmailVerificationOnInvitation` from §4.1 keeps the security floor — invitee must verify email before accept proceeds).
- **Signed in, invitation valid + matches their email:** card centered on page. Title: "Join ?" Subhead: "invited you as ." Body: a brief "What this means" — three bullets covering what membership gives them (operator decides copy). Two buttons: **Accept invitation** (primary), **Decline** (secondary text button).
- **Signed in but invitation is for a different email:** error card — "This invitation was sent to . You're signed in as . Sign out and sign in with the right account?" with a Sign-out button.
- **Invitation expired / cancelled / accepted:** error card with the appropriate message and a "Go to home" link.

Accept calls `auth.api.acceptInvitation`, which creates the member row and (per BA's behavior) typically sets the new org as active for the session. Redirect to `/` (or `/orgs/:slug` if we have one — placeholder for now).

 **Files:** `apps/identity/src/routes/orgs.accept.$invitationId.tsx`, `apps/identity/src/components/invitation-accept-card.tsx`, `apps/identity/src/lib/invitation.functions.ts`.  **Verify:** full flow from O-8's invitation email through accept; member row appears with `role` matching what operator chose; `session.active_organization_id` is set to the new org's id.

At this point the operator can manually onboard a new brand admin end-to-end. They still see no per-org chrome, no theme, no scoping — just a `/orgs/:slug` placeholder. Chat already works (O-2 unblocked brand-admin checks). The next phase makes the active org visible across apps.

Phase 3 — Active org visible to all apps via the envelope

O-10 — Envelope carries `activeOrg`

Extend `EnvelopePayload` with `activeOrg: { id, slug, role } | null`. Bouncer's `toEnvelopeSession` projection in `services/bouncer/src/session.ts` reads `session.activeOrganizationId` from the BA `getSession` response, joins to `organization.slug` and the user's `member.role`. Verifier's result type updates. **Existing apps don't have to consume it yet** — the field is optional and missing on old envelopes is fine.

For dev: when `ENVIRONMENT === "development"` and no bouncer is in front (dev-direct topology), the kit's `getActiveOrg()` falls back to calling `getSession()` and reading the active org from there, same as `getSession()` itself does in dev.

 **Files:** `packages/auth/src/envelope/types.ts`, `packages/auth/src/envelope/verify.ts`, `services/bouncer/src/session.ts`, `services/bouncer/src/envelope.ts`.  **Verify:** bouncer's canonical log line for a signed-in request shows `active_org_id` and `active_org_role`. Decode the envelope's payload manually (paste `base64url` chunks into `jq`); fields are present.

O-11 — Kit accessor `platform.getActiveOrg()`

Surface the envelope's `activeOrg` as a first-class accessor on the object returned by `createPlatformStartApp`. Identity's header gains a **minimal org switcher**: a dropdown listing the user's memberships + pending invitations, with a "Manage orgs" link to `/admin/orgs` for operators. Hidden entirely for solo users (zero memberships, no pending invites).

🔧 **Files:** `packages/kit/src/react-start/...` (the `createPlatformStartApp` factory),
`apps/identity/src/components/header/org-switcher.tsx`, `apps/identity/src/components/header/site-header.tsx`. 🧐

Verify: as a brand admin who is a member of two orgs (seed the second via `/admin/orgs/new` as operator first), the switcher shows both, picking one calls `setActive` and the header updates to reflect. As a solo user (operator-only, no memberships), the switcher is hidden.

You can demo from here. Sign in as operator; create org Acme with alice@acme.com as owner; invite bob@acme.com as member; alice and bob sign in and accept; they see Acme in their switcher. No theme yet, but the manual onboarding loop is complete.

Phase 4 — Chat consumes active org (unblocks chat-spec CP-15 + CP-16)

O-12 — Chat reads active org, gates `createOrgRoom`

Chat's server functions get `getActiveOrg` calls where appropriate. The `createOrgRoom({ orgId, ... })` server fn (chat-spec §1.2's example) verifies `getOrgRole({ userId: env.actor.id, orgId })` is `admin` or `owner` before creating. Public room list shows org rooms whose `ownerOrgId` matches the user's active org.

This is the chat-spec CP-15/CP-16 work; the multi-tenancy work here is just providing the accessors. Chat owns the implementation.

🔧 **Files:** `apps/chat/src/lib/rooms.functions.ts` (new `createOrgRoom`), `apps/chat/src/routes/rooms.tsx` (filter by `activeOrg`). 🧐 **Verify:** as Acme admin (alice), create an org room; bob (Acme member) sees it in his room list; carol (not in Acme) does not. Per chat-spec CP-16 verify steps.

Phase 5 — White-label

O-13 — Theme storage + `getOrgTheme` helper

Type the `organization.metadata` column as `OrgMetadata` (per §3.3) in `services/guestlist/src/schema.ts`. Add the `getOrgTheme` helper to `createGuestlistClient`. **No editor UI yet** — operator can hand-edit theme JSON in D1 to test the read path.

🔧 **Files:** `services/guestlist/src/schema.ts`, `services/guestlist/src/client/guestlist.ts`. 🧐 **Verify:** UPDATE `organization SET metadata = json_set(metadata, '$.theme.wordmark', 'Acme')` WHERE `slug = 'acme'` in local D1; from an app server fn, `getOrgTheme({ orgId: 'org_acme' })` returns the parsed object.

O-14 — `OrgThemeStyle` component + app-root injection

The CSS-var override layer. New component `packages/ui/src/components/org-theme-style.tsx` renders a `<style>` tag with the org's color/typography overrides for `:root` and `[data-theme="dark"]` scopes. App root layouts (identity, chat) inject it during `beforeLoad`. Theme values pass through a strict CSS-safe encoder (MT5) — reject anything containing `;`, `}`, `/`, `*`, `<`, etc.

Wordmark/logo override stays for O-15; this checkpoint is colors only.

🔧 **Files:** `packages/ui/src/components/org-theme-style.tsx`, `packages/ui/src/lib/css-value-safe.ts`, both apps' `src/routes/__root.tsx`. 🧐 **Verify:** hand-edit a `--color-primary` override into Acme's metadata via `UPDATE organization ...`; sign in as alice; primary-colored elements (buttons, focus rings) flip to the new color in both light and dark mode; sign out and back in as a budtender with no Acme membership — platform default returns.

O-15 — Org logo via roadie

Mirror the avatar three-step flow. Roadie's `application.resourceType` gets `"org-logo"`. Guestlist gains `/api/org-logo/register` + `/api/org-logo/confirm` + the public `/u/logo/:refId` read broker. The shared `Logo` component reads from a new `TenantBrandProvider` (`packages/ui/src/tenant-brand.tsx`) populated by app root layouts from the active org's theme.

🔧 **Files:** `services/guestlist/src/index.ts` (mirror of the avatar routes), `packages/ui/src/components/ui/logo/logo.tsx` (consume provider), `packages/ui/src/tenant-brand.tsx`, both apps' root layouts. 🧑🧑 **Verify:** hand-edit `organization.metadata.theme.logoRefId` to a real roadie reference (upload one out-of-band first); the header logo swaps; users without an active org still see the greenroom default.

O-16 — Theme editor UI in identity `/orgs/:slug/theme`

Org-admin route (gate: `member.role ∈ {admin, owner}`). Three sections: color editor (token-name list + color pickers, live preview), logo uploader (drag-drop SVG → registers via the O-15 routes), wordmark text input. Save calls `auth.organization.update({ data: { metadata } })`.

🔧 **Files:** `apps/identity/src/routes/orgs.$slug.theme.tsx`, `apps/identity/src/lib/theme-editor.functions.ts`. 🧑🧑 **Verify:** as alice (Acme owner), pick a color, save; reload; new color sticks. As bob (Acme member, no theme:update), the route shows a "forbidden" empty state.

Phase 6 — Polish & routing

O-17 — OG images respect tenant theme

`OgBrand` accepts `orgId`. Invite landing pages, org-scoped marketing surfaces render brand-aware OG cards. OG renderer authenticates to guestlist via service binding (no cookies; per MT9).

🔧 **Files:** `apps/identity/og/_brand.tsx`, `apps/identity/og/opengraph.og.tsx` (and analogous for twitter, icon). 🧑🧑 **Verify:** paste an Acme invite URL into Slack; unfurl shows Acme's wordmark and colors, not greenroom defaults.

O-18 — Path-per-org bouncer routes

`services/bouncer/wrangler.template.jsonc` `ROUTES` gains `/o/:slug/...` rules. Apps' root layouts reconcile the URL slug against `envelope.activeOrg.slug`; on mismatch, call `setActive` (only if user is a member of the URL's org; otherwise show a 403 page).

🔧 **Files:** `services/bouncer/wrangler.template.jsonc`, `scripts/render-wrangler.ts` (if new tokens needed), both apps' `src/routes/__root.tsx`. 🧑🧑 **Verify:** as alice with `active=Beta`, visit `/o/acme/chat`; the URL flips the active org to Acme; the page renders with Acme chrome.

End-state at O-18: operator-driven org provisioning, full white-label across all apps, chat scoping per-org, OG image branding, path-based org URLs. Demo-ready for a paying customer. Subdomain/custom domain/SCIM are still ahead but not required to ship.

Phase 7 — Deferred

The items below have clean specs (§10.2, §10.3, §11, §6.3, §6.4) but no near-term forcing function. None blocks anything in O-0..O-18.

O-19 (deferred) — Subdomain-per-org

CF Custom Domain registration plumbing. Bouncer host-match rules. Cookie domain scoping verification. Migration runbook: from path to subdomain URLs.

O-20 (deferred) — Custom domains

`tenant_domain` table + bouncer lookup at envelope mint. Operator UI for registering domains. CF Custom Domain provisioning automation.

O-21 (deferred) — SCIM

Per §11.4 (O-SCIM-1..3).

O-22 (deferred) — Dynamic AC

Flag-flip the BA plugin's `dynamicAccessControl.enabled`. Run the `organizationRole` migration. Add a UI in identity for org admins to define roles.

O-23 (deferred) — Teams

Flag-flip the plugin's `teams.enabled`. Migration. UI in identity for managing teams. Update §2 to incorporate teams into the domain model.

O-24 (deferred) — Self-signup for orgs

Flip `allowUserToCreateOrganization` to a function gating on a feature-flag or plan. UI in identity for self-provisioning. Billing integration if applicable.

§13 — Invariants

The checks every multi-tenant code path is expected to honor. Numbered so test names and code comments can reference them stably (analogous to `docs/chat-spec.md` 's I1..I12).

#	Invariant	Enforced at
MT1	<code>envelope.actor === null ⇒ envelope.activeOrg === null</code>	bouncer minter; envelope verifier
MT2	<code>envelope.activeOrg.id</code> , when present, refers to an org the actor is a member of	bouncer minter (sources from session row, which BA's plugin keeps consistent)
MT3	Org-admin writes re-verify role via guestlist <code>getOrgRole</code> ; envelope role claim is never trusted for writes	every server fn that mutates org-scoped data
MT4	<code>member.role</code> ⊥ app-internal roles. An org admin has no implicit chat-room admin (must be bootstrapped per-app)	each app's authz model
MT5	Theme overrides cannot reach raw HTML (XSS via CSS-value injection). All values pass through a CSS-safe encoder before serializing into <code><style></code> blocks	<code>OrgThemeStyle</code> component; theme editor server fn
MT6	A user can only set <code>activeOrganizationId</code> to an org they are a member of	BA org plugin's built-in check; double-checked by identity's switcher route
MT7	SCIM bearer tokens are scoped to one org; SCIM operations only see and mutate that org's data	guestlist SCIM handler (when shipped)
MT8	Custom domain → org binding (<code>tenant_domain</code> table) is operator-only to set, immutable by org admins (prevents takeover)	guestlist <code>tenant_domain</code> write path (operator role required)
MT9	OG image rendering for org-scoped pages fetches theme via service binding, not via cookies (no per-request auth context available)	<code>apps/identity/og/_brand.tsx</code> and analogous OG routes
MT10	Invite emails are sent through promoter, never directly from guestlist to Resend; <code>RESEND_API_KEY</code> stays single-holder	guestlist <code>sendInvitationEmail</code> callback
MT11	A new org's theme inherits platform defaults until an admin explicitly overrides a token (no automatic theme generation)	<code>organization.metadata.theme</code> is sparse; missing keys mean "inherit"
MT12	Org metadata writes pass through BA's <code>update</code> API (which fires <code>beforeUpdateOrganization</code> hooks), never direct DB writes — so audit hooks fire and validation runs	every server fn that touches <code>organization.metadata</code>

§14 — Open questions

Decisions to make before the corresponding checkpoint:

Q	Where it surfaces	Default to assume until resolved
Self-signup for orgs, or operator-only?	O-1 (<code>allowUserToCreateOrganization</code>)	Operator-only in v1. Self-signup behind a feature flag later.
One org per user vs many? Cap on memberships?	O-1 / O-2	Many. No cap. Org switcher always present.
Theme overrides validated against a strict allowlist or accept any CSS token?	O-14	Strict allowlist = shadcn semantic token surface only (§8.1.3). Reject unknown keys. Reject values that aren't valid CSS color/length tokens. Greenroom's internal palette names are intentionally not on the allowlist.
Theme version pinning (so a shadcn token rename breaks gracefully)?	O-13	<code>organization.metadata.theme.schemaVersion: 1</code> (required field on <code>TenantTheme</code>). Migrations on bump rewrite stored blobs.
Cross-org notifications (an org admin gets an event in another org)?	O-4	Not in v1. Each app's notification surface is org-scoped.
Custom domain → org binding stored where? Separate table or <code>organization.metadata.domains[]</code> ?	O-20	Separate <code>tenant_domain</code> table with FK to org. Easier to index and enforce uniqueness.
Operator audit log (who created which org, who flipped which theme)?	O-1+	Reuse the existing canonical-log infrastructure with structured events; promote to a dedicated <code>audit_log</code> table when retention requirements appear.
What's the right move when a user is removed from an org while connected to its chat room?	O-4	Defer to chat's existing membership-revocation path (already in scope per <code>docs/chat-spec.md §moderation</code>). Org membership and room membership diverge cleanly.
Retailers as a second org type? Same <code>organization</code> table or separate?	Future (post-v1)	Same table, add <code>organizationType: "brand" "retailer"</code> discriminator on <code>organization</code> . Retailers get a different role set (retailer-owner, store-manager, budtender-staff?) and a different relationship to chat (retailer-owned rooms scoped to a store's staff). v1 ignores this entirely; the schema choice is so additive it won't constrain.

§15 — Threat model snapshot

Higher-fidelity threat modeling is per-feature (the SCIM rollout will get its own STRIDE pass); this is the cross-cutting set every contributor should keep in mind.

- Cross-org data leak via envelope misuse** — apps that trust `envelope.activeOrg.role` for writes give a user 30s of stale-admin power on demotion. Mitigation: MT3. Apps that mutate **always** re-verify.
- Theme XSS** — tenant supplies `--color-primary: red; } body { display: none; --:` in the metadata. `OrgThemeStyle` must encode values strictly. Mitigation: MT5. Theme editor's server-side validation also rejects values containing `;`, `}`, `/*`, or non-CSS-token characters, and rejects keys not on the `SHADCN_TENANT_TOKEN_ALLOWLIST` (§8.1.3).
- Custom-domain takeover** — attacker registers `chat.victim.com` as their CF Custom Domain. Mitigation: MT8 + manual operator approval on custom-domain provisioning until automated DNS verification is in place.

4. **SCIM token leak** — token leaked from a tenant's Okta config grants create/delete on all their members. Mitigation: per-org scoping (MT7) + rate limiting (BA `apikey` plugin handles per-key rate limits) + per-key audit log.
5. **Invite email spoofing** — attacker triggers a flood of legitimate invites to embarrass a tenant. Mitigation: `invitationLimit` per inviter; `cancelPendingInvitationsOnReInvite` prevents accumulation; per-org rate limit on `inviteMember` (BA plugin supports via `beforeCreateInvitation` hook throwing on overrate).
6. **Wordmark / logo for phishing** — tenant uploads "Bank of America" wordmark, sends invites. Out-of-band: T&S review on org provisioning, takedown process via the operator UI. Not a code-level mitigation.

Appendix A — Migration plan

Going from "single-tenant" to "org-aware" is small if done in order.

1. **PR 1** — Add org plugin and migration. No app consumes it yet. Plugin tables exist in D1; the bouncer envelope is unchanged. Safe to ship independently.
2. **PR 2** — Add bouncer envelope's `activeOrg` field. Verifier accepts both shapes (old envelopes have no `activeOrg`; verifier defaults to `null`). Bouncer mints with `activeOrg: null` until guestlist returns a session with `activeOrganizationId` — which only happens once the plugin is querying live data.
3. **PR 3** — Identity's org switcher + admin UI. Seeded orgs for dev. Both Acme and Beta exist in local D1; the same dev user is a member of both.
4. **PR 4** — Chat picks up `activeOrg` (unblocks chat-spec CP-15+CP-16).
5. **PR 5** — Tenant theme override path (`OrgThemeStyle` , theme editor, org logo via roadie).
6. **PR 6** — OG images consume tenant theme.
7. **PR 7** — Bouncer ROUTES adds path-per-org rules. Identity reconciles slug → `setActive`.

Each PR is independently rollback-safe; nothing in flight depends on the next.

Appendix B — Glossary

- **Active organization** — the org currently in scope for a session. Stored on the BA `session.activeOrganizationId` column; mirrored into the bouncer envelope as `activeOrg`. Per-session, not per-user.
- **Member** — the BA org plugin's join row tying a user to an org with a role. Distinct from app-internal membership (e.g. chat's room membership).
- **OrgMetadata** — typed shape of `organization.metadata` in our schema. Carries `theme`, `plan`, `flags`, `operatorNotes`. Extension point.
- **Operator** — greenroom platform staff. Identified by `user.role === "admin"`. Has powers over all orgs; orgs have no power over operators.
- **Path-per-org** — URL strategy `greenroom.app/o/<slug>/...`. v1 default.
- **SCIM** — System for Cross-domain Identity Management. The protocol enterprise IdPs (Okta, Entra) use to provision and deprovision users on third-party SaaS.
- **Subdomain-per-org** — URL strategy `<slug>.greenroom.app/...`. Designed-for but not v1 default.
- **TenantTheme** — typed shape of `organization.metadata.theme`. Light/dark CSS variable overrides + logo ref + wordmark.
- `tenant_domain` — (future) table mapping custom domain → orgId. Powers `chat.acme.com` deployments.
- **White-label** — the ability for a brand to present greenroom as their own product to their staff: their colors, their logo, their wordmark, optionally their domain.